INDEX

S. No	Topic	Page No.
	Week 1	
1	Biological Neuron	1
2	From Spring to Winter of AI	8
3	The Deep Revival	18
4	From Cats to Convolutional Neural Networks	25
5	Faster, higher, stronger	29
6	The Curious Case of Sequences	31
7	Beating humans at their own games (literally)	38
8	The Madness (2013-)	41
9	(Need for) Sanity	50
10	Motivation from Biological Neurons	54
11	McCulloch Pitts Neuron, Thresholding Logic	62
12	Perceptrons	73
13	Error and Error Surfaces	82
14	Perceptron Learning Algorithm	85
15	Proof of Convergence of Perceptron Learning Algorithm	98
	Week 2	
16	Deep Learning(CS7015): Linearly Separable Boolean Functions	108
17	Deep Learning(CS7015): Representation Power of a Network of Perceptrons	113
18	Deep Learning(CS7015): Sigmoid Neuron	124
19	Deep Learning(CS7015): A typical Supervised Machine Learning Setup	136
20	Deep Learning(CS7015): Learning Parameters: (Infeasible) guess work	146
21	Deep Learning(CS7015): Learning Parameters: Gradient Descent	157
22	Deep Learning(CS7015): Representation Power of Multilayer Network of Sigmoid Neurons	179
	Week 3	
23	Feedforward Neural Networks (a.k.a multilayered network of neurons)	209
24	Learning Paramters of Feedforward Neural Networks (Intuition)	222
25	Output functions and Loss functions	229
26	Backpropagation (Intuition)	244
27	Backpropagation: Computing Gradients w.r.t. the Output Units	255
28	Backpropagation: Computing Gradients w.r.t. Hidden Units	267
29	Backpropagation: Computing Gradients w.r.t. Parameters	279
30	Backpropagation: Pseudo code	287

31	Derivative of the activation function	292
32	Information content, Entropy & cross entropy	294

Week 4

33	Recap: Learning Parameters: Guess Work, Gradient Descent	312
34	Contours Maps	324
35	Momentum based Gradient Descent	333
36	Nesterov Accelerated Gradient Descent	346
37	Stochastic And Mini-Batch Gradient Descent	355
38	Tips for Adjusting Learning Rate and Momentum	368
39	Line Search	377
40	Gradient Descent with Adaptive Learning Rate	383
41	Bias Correction in Adam	404

Week 5

42	Eigenvalues and Eigenvectors	412
43	Linear Algebra : Basic Definitions	424
44	Eigenvalue Decompositon	434
45	Principal Component Analysis and its Interpretations	442
46	PCA : Interpretation 2	465
47	PCA : Interpretation 3	477
48	PCA : Interpretation 3 (Contd.)	482
49	PCA : Practical Example	484
50	Singular Value Decomposition	491

Week 6

51	Introduction to Autoncoders	510
52	Link between PCA and Autoencoders	543
53	Regularization in autoencoders (Motivation)	556
54	Denoising Autoencoders	564
55	Sparse Autoencoders	582
56	Contractive Autoencoders	590

Week 7

57	Bias and Variance	599
58	Train error vs Test error	608
59	Train error vs Test error (Recap)	624
60	True error and Model complexity	631
61	L2 regularization	639
62	Dataset augmentation	656

63	Parameter sharing and tying	660
64	Adding Noise to the inputs	662
65	Adding Noise to the outputs	667
66	Early stopping	671
67	Ensemble Methods	679
68	Dropout	686

Week 8

69	A quick recap of training deep neural networks	700
70	Unsupervised pre-training	706
71	Better activation functions	726
72	Better initialization strategies	752
73	Batch Normalization	774

Week 9

74	One-hot representations of words	786
75	Distributed Representations of words	793
76	SVD for learning word representations	802
77	SVD for learning word representations (Contd.)	803
78	Continuous bag of words model	814
79	Skip-gram model	839
80	Skip-gram model (Contd.)	846
81	Contrastive estimation	856
82	Hierarchical softmax	861
83	GloVe representations	871
84	Evaluating word representations	877
85	Relation between SVD and Word2Vec	885

Week 10

86	The convolution operation	888
87	Relation between input size, output size and filter size	907
88	Convolutional Neural Networks	919
89	Convolutional Neural Networks (Contd.)	935
90	CNNs (success stories on ImageNet)	951
91	CNNs (success stories on ImageNet) (Contd.)	972
92	Image Classification continued (GoogLeNet and ResNet)	974
93	Visualizing patches which maximally activate a neuron	989
94	Visualizing filters of a CNN	993
95	Occlusion experiments	999

96	Finding influence of input pixels using backpropagation	1003
97	Guided Backpropagation	1008
98	Optimization over images	1012
99	Create images from embeddings	1022
100	Deep Dream	1032
101	Deep Art	1039
102	Fooling Deep Convolutional Neural Networks	1044

Week 11

103	Sequence Learning Problems	1050
104	Recurrent Neural Networks	1057
105	Backpropagation through time	1067
106	The problem of Exploding and Vanishing Gradients	1080
107	Some Gory Details	1089
108	Selective Read, Selective Write, Selective Forget - The Whiteboard Analogy	1095
109	Long Short Term Memory(LSTM) and Gated Recurrent Units(GRUs)	1103
110	How LSTMs avoid the problem of vanishing gradients	1126
111	How LSTMs avoid the problem of vanishing gradients (Contd.)	1133

Week 12

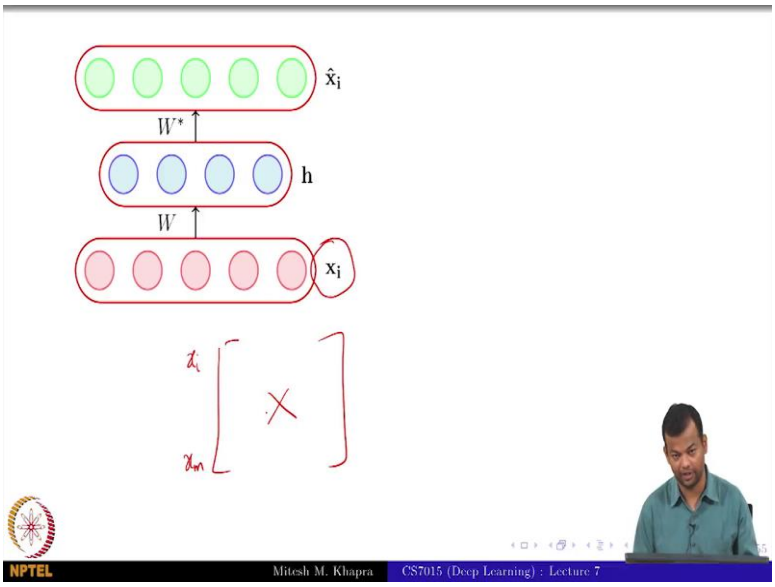
112	Introduction to Encoder Decoder Models	1152
113	Applications of Encoder Decoder models	1169
114	Attention Mechanism	1185
115	Attention Mechanism (Contd.)	1205
116	Attention over images	1207
117	Hierarchical Attention	1216

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture - 07
Autoencoders and relation to PCA, Regulation in autoencoders, Denoising autoencoders, Sparse autoencoders, Contractive autoencoders

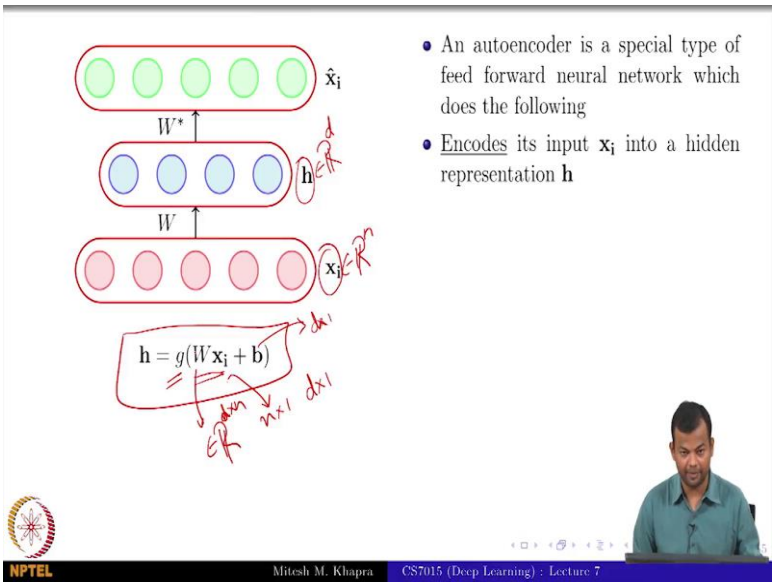
Welcome to lecture 7 of the course on Deep Learning CS 7015. In this lecture we are going to talk about auto encoders and we will focus on their relation with PCA. Then talk about regularization in auto encoders, wherein we will look at denoising auto encoders, sparse auto encoders and contractive auto encoders. So, let us begin with the introduction to auto encoders what they are.

(Refer Slide Time: 00:36)



So, this is what a typical auto encoder looks like. And as you can see this is very much like a feed forward neural network you have an input which is x i. So, you are given some training data you are given some i samples x i to x n. So, this is your training matrix x which we have seen in the previous lectures. So, this is one of those training inputs x i, and then you have a hidden layer and then an output layer. So, let us look at what is the configuration of the hidden layer and what does the output layer actually try to do.

(Refer Slide Time: 01:08)



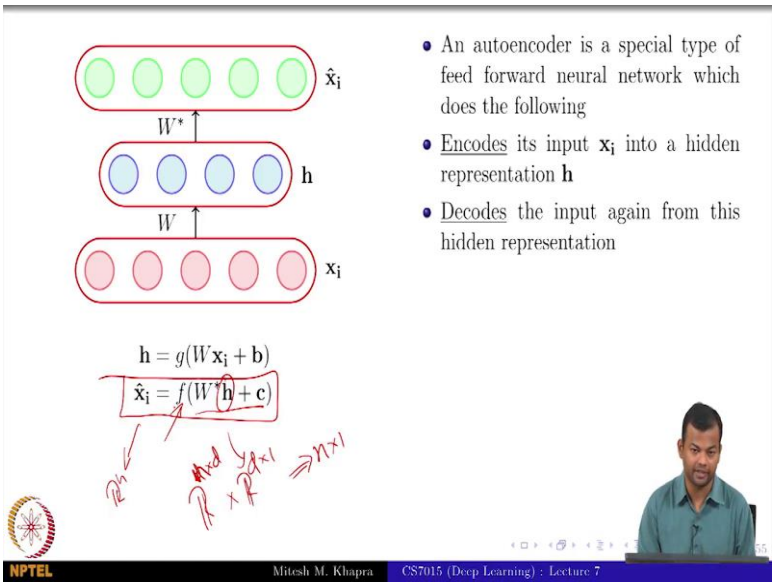
So, it is a very special type of a feed forward neural network. What it does is it encodes with input x i to a hidden representation h ok, and it uses an encoded function to do this. So, this is what the encoded function does. It first does a linear transformation.

So, W is a matrix and x i is a vector and you again have the bias b as a vector right? So, let us look at these dimensions right, so let us try to fix some dimensions. So, suppose x i belongs to R n that is what we have been considering throughout the course. So, far and let us say h belongs to R d.

So, it is a d dimensional representation. So in that case what would W be yeah. So, W would be R n cross or the d cross n right? So, it will multiply with the n cross 1 vector which is x i and give you a d cross 1 output right? And similarly the b would also be d cross 1, and then on top of that you have this non linearity g, which will be operating at element wise just as we had seen earlier. So, it could be any of the sigmoid functions the logistic or tanh and so on.

So, the end result is you have taken an input x i and encoded into a hidden represent h by using a linear transformation first, and then a non-linear transformation right? So, I refer to W x plus b as a linear transformation, because it is a matrix multiplication. Now once you have constructed this hidden representation.

(Refer Slide Time: 02:41)



The job of the decoder or the latter half of the feed forward neural network which is this half is to take this encoded representation. And then try to reconstruct x again from it.

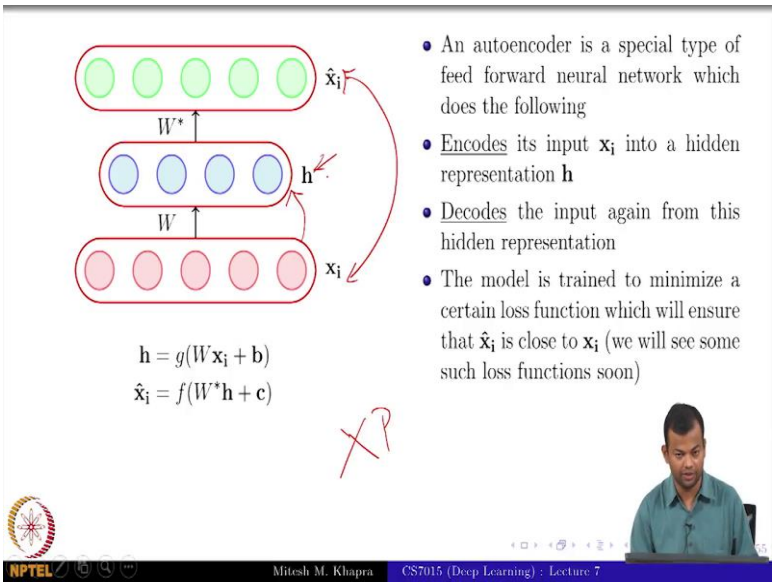
So, again let us first look at the equation. So this is the equation for the decoder, where again you first take the hidden representation do a linear transformation and then you again have some function on non-linearity on top of it right? So, we will see what this function can be so we will refer to it as f for, now we will not say whether this is sigmoid or linear or what kind of a function it is, we will come back to it later on.

So, now let us again look at these dimensions. So what is x i? x i is again R n and your h was R d. So, you have to go from a d dimensional input to an n dimensional output. So, again your W star is going it to be R d cross sorry R n cross d. So, it will multiply with a d cross 1 vector and give you an n cross 1 output right? And that will pass through some function and it will give you x i hat which is a reconstruction of x i.

So, why are we trying to do this right we took an input x i, we computed it is hidden representation by doing some non-linear and linear transformation and then again we are trying to reconstruct x i hat. So, why are we trying to do this? So reason we are doing this is that we want to learn, what are the most important aspects or most important characteristics of the input data x i right? So, if you compute a hidden representation H, which is presumably smaller than your original input data.

And from that hidden representation if you are able to reconstruct x i right? Then that would mean that this hidden representation captures everything that is required or everything that is yeah everything that is required to reconstruct x i from x i from the original input right?

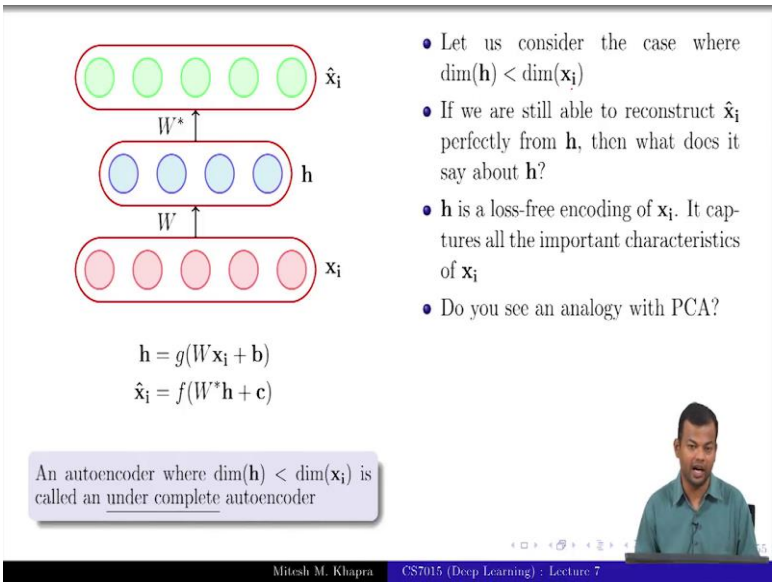
(Refer Slide Time: 04:37)



So, the model will be trained to minimize the difference between x i and x i hat. So, you want to make sure that after passing through this bottleneck which is the hidden representation you are able to reconstruct x i, and the reconstructed output is very close to the original input right?

So, can you see an analogy with PCA, where you are trying to find this hidden representation or this most important elements of the original input x i. So, there we had used this linear transformation where we are taking the original input x. And transformed it to a new basis and we had used that basis for representing the original input right? So, something similar is happening here, we are using this hidden representation h to represent our original input.

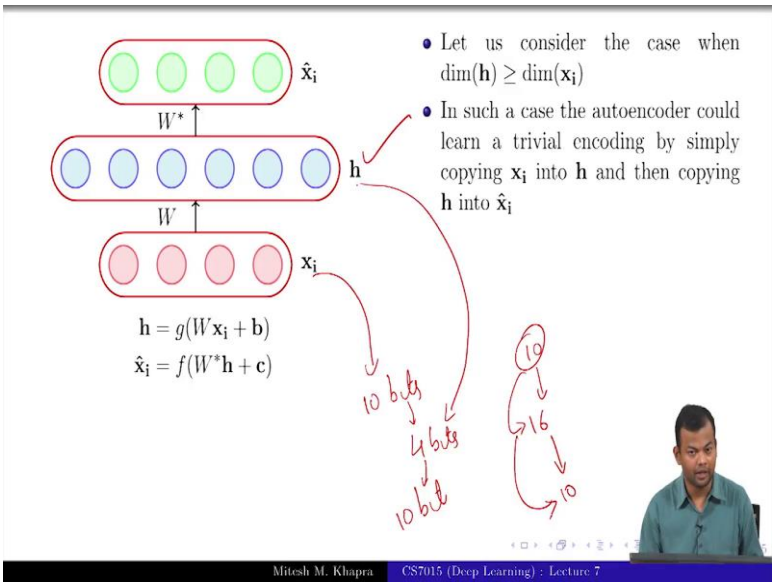
(Refer Slide Time: 05:22)



Now, let us consider a few cases the first cases when the dimension of h is less than the dimension of x i. In this case as I was trying to say earlier if we are still able to reconstruct x i hat perfectly from h. Then what does it say about h it tells us that h is a loss free encoding of x i. It captures all the important characteristics of x i write just repeating what I had said on the previous slide. And now you can see an analogy with PCA because h has all the important characteristics required from the original input data.

So, it has probably got rid of all the noise or all the low variance dimensions or the correlated dimensions and so on. And this is just the compact representation, which is as good as the original representation and from there you can reconstruct the original representation. And such an auto encoder where the dimension of the hidden representation is less than the dimension of your original input is known as an under complete auto encoder.

(Refer Slide Time: 06:17)



Now, let us look at the other case where the dimension of the hidden representation is greater than the dimensional of the original input, ok such an auto encoder is I will tell you what it is called. So, we will we are looking at the case where the dimension of the hidden representation is greater than the dimension of the original input.

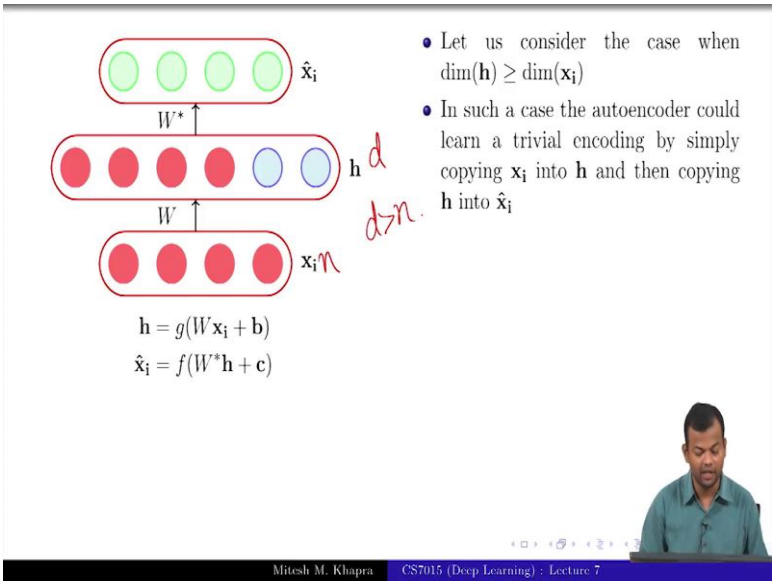
So, now in such a case the auto encoder could learn a very trivial encoding by simply copying x i into h and then copying h into x i right, so think of this from a compression point of view right? So, now, suppose you have 10 bits initially right, and then you want to somehow compress it and store it only in 4 bits.

And now this 4 bits should be such that it captures everything that was there in the original 10 bits because you would want to reconstruct the original input again right? So, this is what we do typically when we compress any of our files right we have a larger file we compress into a smaller information while making sure that everything important is there. So, that whenever I want to recover it, I can just recover it from there.

So, this is definitely a hard task, but now what I am doing in this auto encoder is that I had 10 bits, I am actually giving it more bits now because the dimension of h is greater than the dimension of the input. And then from these 16 bits, I want to reconstruct the 10 bits now this is a very trivial task right because all I could do is copy these 10 bits into the first 10 bits here leave the remaining 6 blank. And then from those 10 bits just

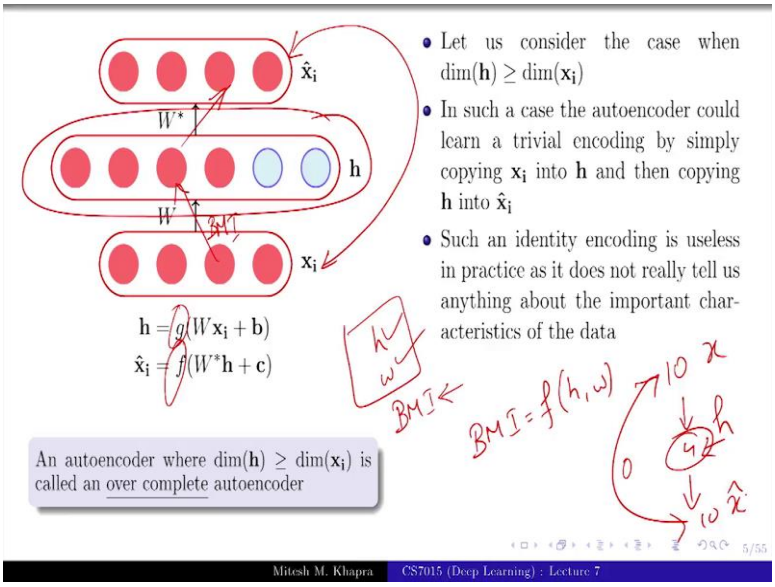
reconstruct the input that is very, very trivial if you give me more storage and what I originally needed, then definitely I can easily reconstructed.

(Refer Slide Time: 07:58)



So, this looks very trivial and this is what it could do right just copy the input to the first the n bits.

(Refer Slide Time: 08:12)



So, this was n and this was d and we are looking at the case where d is greater than n. So, it will just copy the input to the first n bits and then just take it back to the output just as i

said in the case of you have 10 bits 16 bits and then again 10 bits it is very trivial to do this.

So, such an identity encoding is useless because you are just not running any important characteristics of the data, your h is almost the same as x i it also has all the useless information that x i had. In fact, it has slightly more because it has these blank units also, but this is not really useful right why would you want to actually learn such a hidden representation right? So, it is not clear why would you want to do that so we will take a look at it we will come back to why this is important.

So, such an auto encoder is known as an over complete auto encoder because it has the hidden representation has more number of neurons as compared to the original input, now let us look at a case where this would actually be important right? So, this is a very rough intuition for why you would want an over complete auto encoder.

So, let us consider the case where you have as input 1 of the features that you are looking as BMI. So, suppose you are trying to find out whether the person is likely to get a certain disease or not right? So, whether he would have a heart attack or whether he would have a diabetes, would have diabetes and so on. And you are looking at various parameters or various medical parameters of that person and one of them could be height one of them could be weight and one of them could be BMI.

Now, for whatever reason you have not computed the height and weight and you have only looked at the BMI. So, now, what has happened in your input and all of you know that BMI is actually body mass index which is a function of the height and the weight.

So, now, what has happened is that in your original input there was already this compact the your feature space is already compact, because you would actually look at you should have actually looked at both the features height and weight, but for some reason you have only computed BMI and you could think of various some other correlated features, which are functions of many other features, but you do not look at all those features and just this final function of those features.

So, now in that case if suppose your prediction is that this person has or has a high likelihood of being of high likelihood of having diabetes at some point in his life. Then

you would want to know whether it was the height, or whether it was the weight which was responsible for this.

So, in your original input your features are actually entangled and you would like to disentangle them right? So, you would want to go from this smaller feature space to a larger feature space where some of these entangled features get is disentangled. So, in those cases we reach an over complete auto encoder; however, the problem still remains that there is no reason why the machine should actually learn to disentangle these features it could still just simply copy the BMI here and then copy it back here.

So, that is why when you are dealing with over complete auto encoders you will have to do something special to prevent this kind of identity encoding. So, as you just take the input and copy it to the hidden layer, and then copy it back to the output. So, we will look at what kind of special treatment you need to do to prevent these kind of identity representations.

(Refer Slide Time: 11:24)

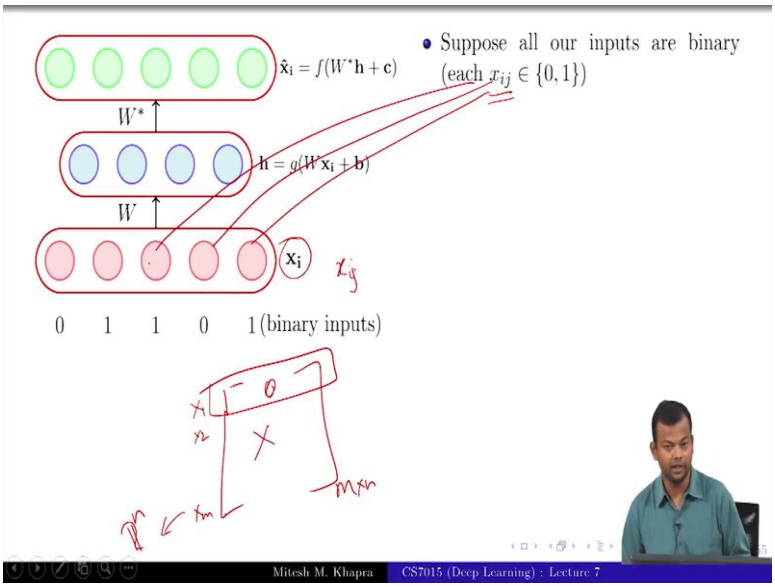


Here is the road ahead. So, first we will talk about the choice of f x i g x i right? So, we did not say anything about what these functions f and g have to be. So, we will talk about those and then we will talk about the loss function. So, I have just told you so far that we will train this model in a way that x i is very close to x i hat right? And I have argued that if we are able to actually achieve that that x i hat is the same as x i in which case

presumably, presumably the loss would be 0. That means, our hidden representation has captured all the important characteristics of the original data.

Same as in the analogy of 10 bits to 4 bits to again 10 bits right? If I am able to reconstruct this without any error that means, loss is 0 then these 4 bits or the hidden representation of my original x was actually able to capture everything that was important in x. So, that it can reconstruct x again as x hat without losing any information right? So, that is the loss function that we would want now what is the actual mathematical formulation for this loss function that is what we will see next.

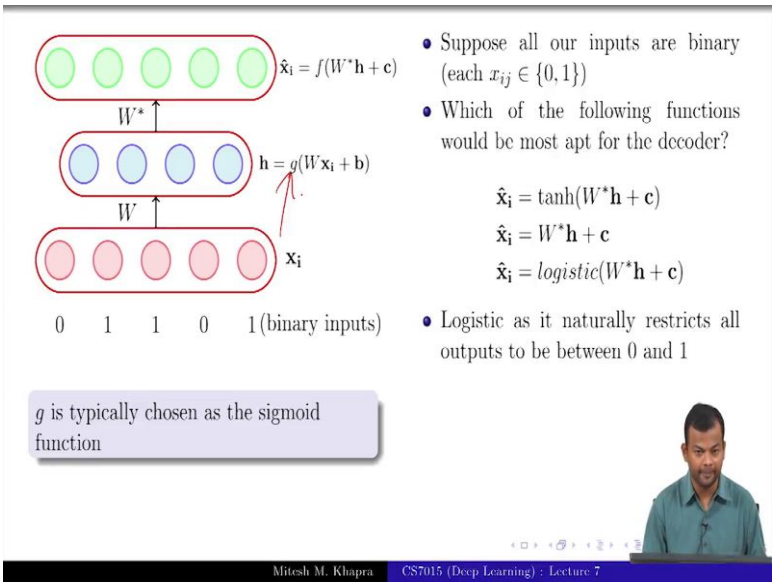
(Refer Slide Time: 12:35)



So, first let us start with the choice of f, f and g. So, we will consider 2 case 2 cases one case when your inputs are binary and the second case when your inputs are actually real numbers right? So, the first we will look at the binary case. So, now, just some notation clarification. So, remember our original data was this matrix x which was m cross n. That means, you had x 1 x 2 up to x n and each of these was R n.

So, now when I am referring to the entire row or entire data instance I will use bold x i, as I have circled here. And I want to refer to one of the elements of this guy, then I will use this notation x i j same as what I have written here. So, what I am saying is that each of these x i j‘s actually is a binary variable.

(Refer Slide Time: 13:25)



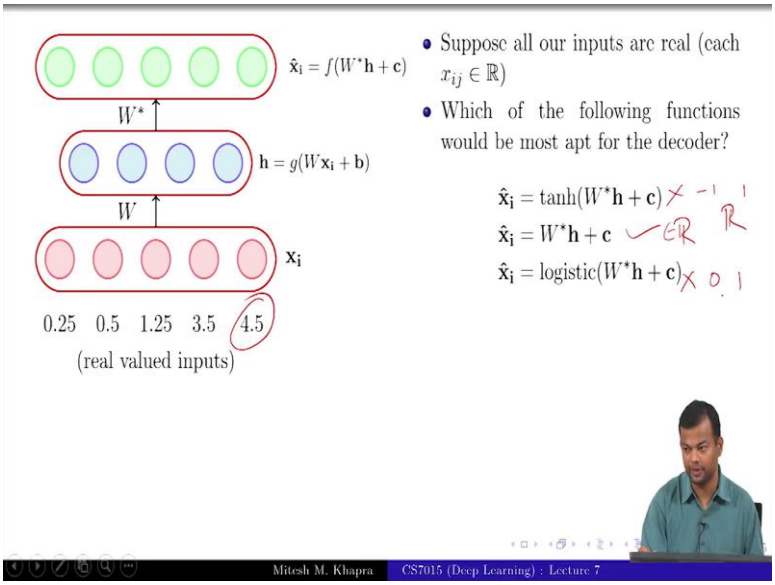
Now, which of the following functions would be most appropriate for the decoder? So, remember was the input was binary. That means, your output also has to be binary you do not want to produce numbers arbitrarily large belonging to any or want do not want to produce any real number you want to produce numbers which lie between 0 to 1. So, in such a case what would be an appropriate loss function or sorry what would be an appropriate function for the decoder. So, remember I am asking you what would f be.

So, I am giving you 3 choices it should be tanh or just a linear decoder or a logistic function. Which of these would be most appropriate logistic why would that be because it will make sure that your outputs are between 0 to 1, tanh would give you outputs between minus 1 to 1. But you do not want that because your inputs were between 0 to 1. So, when you are reconstructing; obviously, you want outputs between 0 to 1, you do not 1 minus 1 to 1. And linear of course, can give you any real number which is not what you want right?

So, if you produce any arbitrary real number like hundred and so on, your loss is going to be very high because your inputs were just 0 to 1 and you are producing these arbitrary real numbers which are very different from what your input was ok. So, in this case the logistic function makes the most appropriate choice. And g is typically that means, the encoded function is typically again chosen as a sigmoid function. So, it could either be

the logistic function or the tan h function right? So, the there is you could choose any of these as the encoder function fine.

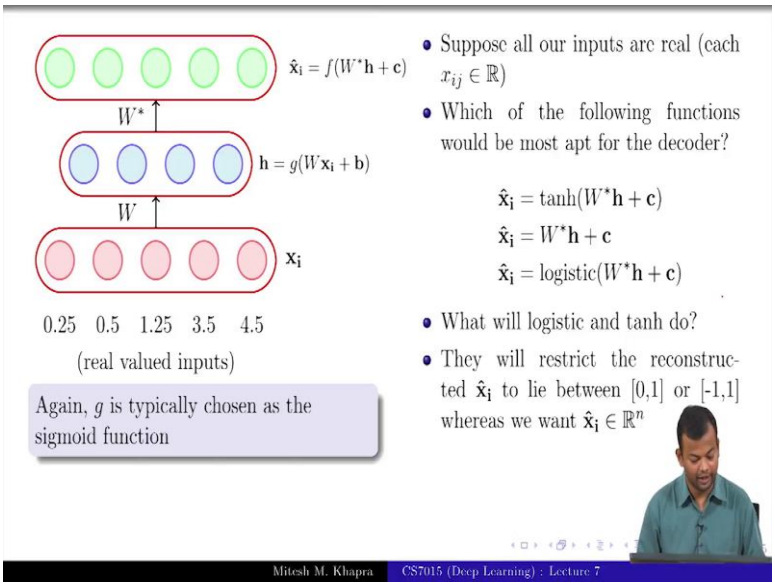
(Refer Slide Time: 15:01)



Now, let us consider the other case where your inputs are real valued. That means, when you reconstruct something you should again produce real values. That means, your function f should take whatever is the input given to it and map it to some real numbers right? So, that is what we want from this function f earlier in the binary case we wanted it to map it to binary numbers right? So, that is the difference that we have now.

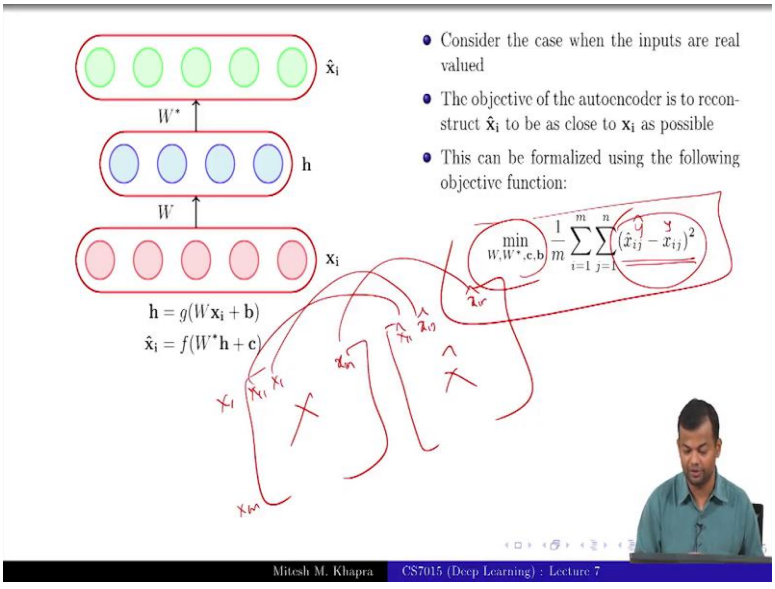
So, in this case which of the following would be appropriate? The second one right because tan h does not make sense because it will just produce minus 1 to 1, but you want to produce any possible real number because some of these are actually higher than 1 greater than 1. Linear would be fine because it will produce any real number logistic is again not fine because it will produce numbers between 0 to 1.

(Refer Slide Time: 15:51)



So, the logistic and tan h as I said would clamp the output to certain ranges. So, that is not appropriate hence you should choose the linear function and again in this case also g is typically chosen as the sigmoid function fine ok. So, the next thing that we look at is the choice of the loss function.

(Refer Slide Time: 16:10)



And again we will consider both the cases, where a case the first case is the inputs are real valued and the second case is when the inputs are binary.

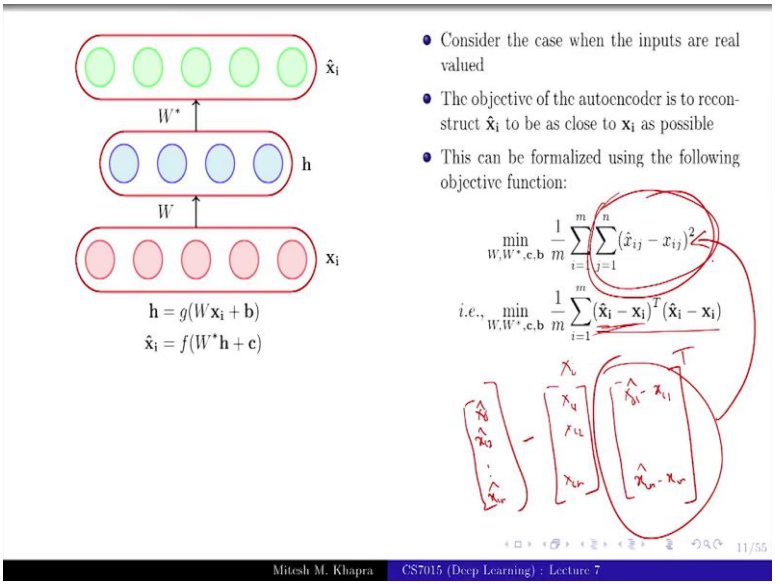
So, let us look at the real case first. Now here the objective of the auto encoder is to reconstruct x i hat to be as close to x i as possible. Now we have actually seen this before, so something similar before when we were talking about regression. So, now, you want to produce real valued outputs and they should match your real valued inputs. So, what is an appropriate loss function that you can choose the squared error loss function right?

So, what does this actually capture? It says that for all my input data x 1 to x m. For each of these dimensions x 1 to up to x 1 n right? I want to make sure that my original input, I will have a similar x hat reconstructed where I will have x 1 one hat x 1 2 hat and x 1 n hat.

So, I want to make sure that each of these pairs of variables are actually similar. And I can capture that by ensuring that the squared error loss between the ij'th entry in my output is the same as this or sorry. Rather I could capture the squared error loss between the ij'th entry in the output and the input ok. That is what this function is trying to capture straightforward similar things we have seen while we were doing regression.

Except that there we had y hat and y, but here we are just trying to reconstruct the input. So, there is no y here we just have the x ok. And the parameters of the objective function are of course, all the variables or all the parameters that we have in a network, which is W W star c and b.

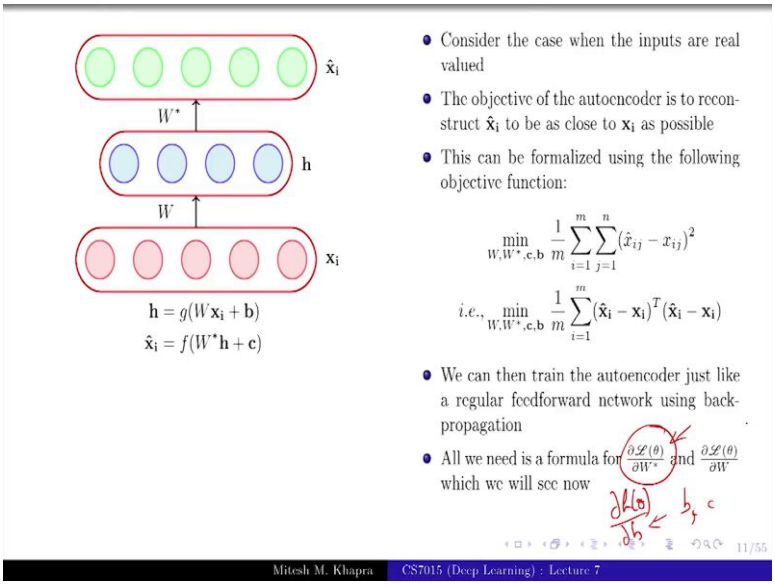
(Refer Slide Time: 17:56)



And the matrix or the vector way of writing this is the following. So, we have x i. So, what I am looking at here is I have gotten rid of this summation and I am just written it in vector form. So, let me just explain what this means. So, this is what x i would look like right? So, this would be x i 1 x i 2 up to x i n ok this is the vector. And then you have the x i hat vector which is going to be x i 1 hat x i 2 hat up to x i n hat.

So, taking the difference between these two vectors that is what this term is. So, what you will get is essentially x i 1 hat minus x i 1 up to x i n hat minus x i n right? And then you are taking the dot product of this vector with itself which will essentially give you this summation right? So, the dot product of this vector with itself is actually going to be this summation, it is going to be the sum of the squares of the elements of this vector and that is exactly what we wanted.

(Refer Slide Time: 19:06)

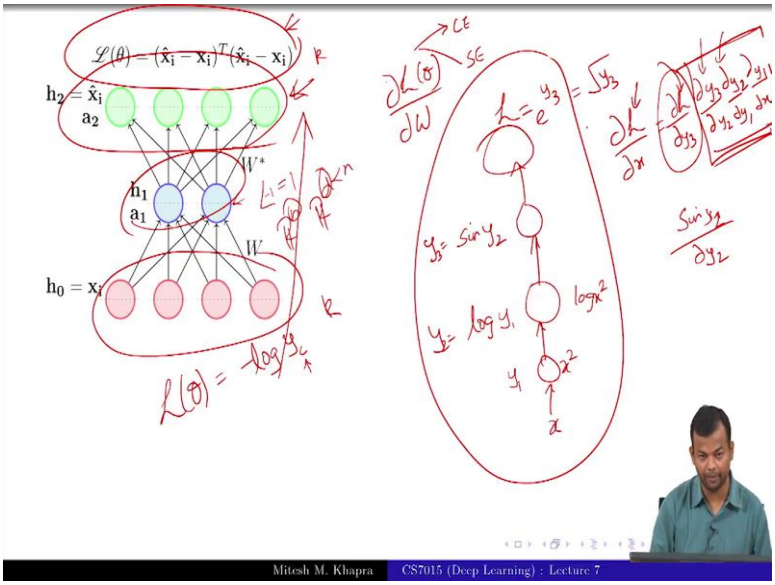


So, this is a more compact vectorial way of writing the same thing. And now we can just train the auto encoder by treating it as a regular feed forward neural network this is just a like any other feed forward neural network you have find the loss function.

And you can just use back propagation to treatment right, but and in this case all we will need is a formula for the gradient of the loss function with respect to with your parameters which are W and W star, I have again ignore the biases and the bias is here b and c. So, we will also need dou l theta by dou b and dou l theta by dou c right? So, these

two gradients also you will need, but these are generally the easier ones to handle if you know how to compute this b and c are very easy.

(Refer Slide Time: 19:48)



So, let us look at this now what we need for back propagation as I said we will need this gradient right? All these four gradients but let us focus on one of these. Now we have already done back propagation and we have looked at arbitrary neural feed forward neural networks here right? We did not have we just said that there are L hidden layers and in this case L is equal to 1 right, or other we had said there was l minus 1 hidden layers and the l'th layer was the output.

So, in this case l minus 1 is equal to 1; that means, there is just 1 hidden layer. So, it does not matter we had actually derived it for the general case when l is equal when the number of hidden layers is l minus 1 and here we just have 1 eight n layer. So, it is much more simpler than what we had learnt. And even for the number of neurons in the each of these layers we are just assumed general that it could be R n. And in this case we would have some R d, which is less than n or it could even be greater than n right?

So but it does not matter because whatever algorithm we had or whatever equations we had derived for back propagation. They did not care about what this n or d was we had just derive it in general terms right, and the same for the output layer.

We did not assume any number of inputs any number of neurons in the output layer we again said that it has some k neurons, but there the catch is in the earlier case when we had derived back propagation. We were dealing with classification and we had these k classes that we want to predict at the output.

And in which case our loss function was actually the cross entropy or the negative log likelihood function right? Where we were trying to maximize the probability of the correct class out of the k given classes, but here our loss function is slightly different it is actually this squared error loss between the input and the output.

So, now given this difference in the loss function does it mean that everything that we learn in the previous lecture on back propagation we just have to throw it all away because now there is a new loss function? That means, my gradients are going to be very different from what I had derived for the back propagation loss, where I was looking at the cross entropy loss as compared to the squared error loss. So, does it mean that, I will have to throw away all the hard work that we had done in that course in that lecture or can we reuse something from them we can reuse.

So, let us look at what we can reuse and I will just give you an intuitive explanation for that. So, you can think of this as a composite function right? And you are taking your input passing it through a lot of functions and then arriving at the output, and then your loss function is actually a function of the output itself.

So, what we have is something like this right we have a situation like this. That you had an input x you computed some function of it say x square right? So, I will call this as y 1 then you computed some other function of it say y 1 say log of y 1 right? So, they this was log of y 1. So, in effect it is actually log of x square because y 1 is equal to x square and then some other function and then finally, you had the output. So, you had this other function which was sign of I am calling this y 2. So, say this was sign of y 2 and finally, you had this function which was e raise to y 3.

So, you have a very complex composite function of your original input right and this is your final output function that you are considering which is e raise to y 3. Now the way you would do this is if you want to take the gradient of d l with respect to your input d x right? In that case what would you do is you just apply the chain rule you will write it as dou l by dou y 3 dou y 3 by dou y 2 dou y 2 by dou y 1 and then dou y 1 by dou x right?

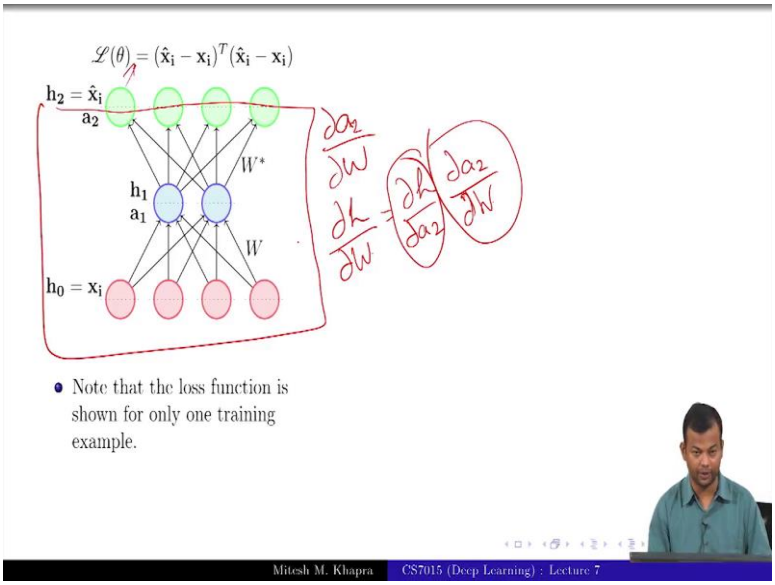
And this is something very similar that we are done in the back propagation lecture we had constructed this chain and then we had attacked every element of this chain and derived how to deal with that right derived an neat expression for that.

Now, the question which I am asking you is that in that lecture we had assumed a certain l. And that l was actually cross entropy, but in this lecture I have actually changed the l, what I am saying is the l is actually equal to the squared error loss. Now does that mean that I have to throw away all this work that I had done no right?

So, even in this example if you look at it suppose I change this function from e raise to y 3 to say square root of y 3. So, I have just changed my l, but notice all of these other guys are going to remain in the same, because y 3 is still sign of y 2. So, that the derivative of y 3 with respect to y 2 is not going to change. Even though I have changed the output function the loss function everything else is going to be remain in the same right?

So; that means, all these portions I could just reuse from the time when I had computed for this chain. I just need to rework on this final expression and plug it in right? So, that is why all the work that we had done in the case of back propagation will not go to waste in particular everything that we had done.

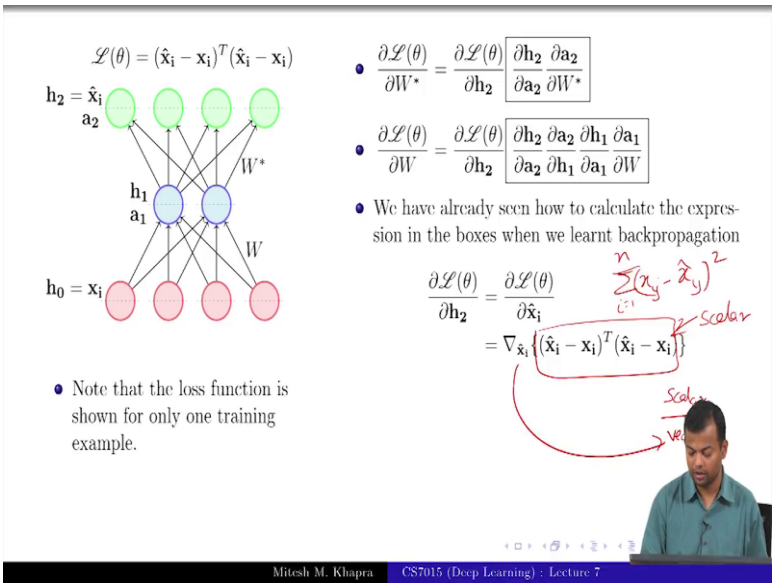
(Refer Slide Time: 24:45)



So, let me just go to the next slide. So, in particular everything that we had done for this portion of the network right which is actually dou a 2 all the way up to dou W right? So, if ok, so let me write it like this I want dou l by dou W. So, I can write it compactly as dou l by dou a 2 and then dou a 2 by dou W right?

So, this portion is not going to change because I am not change any of the functions here, I have just assumed sigmoid or logistic or the same kind of network. The only thing I have changed is something at the output layer. So, I will just need to recomputed this and the rest of it can be reused right? So, that is the intuition which I wanted to give you.

(Refer Slide Time: 25:27)



And that is exactly what is written on this slide. So, I am written it as dou l theta by dou W star that is the first gradient I have interested in. And I could write it as dou l theta by dou h 2 dou h 2 and dou a 2 by W star right?

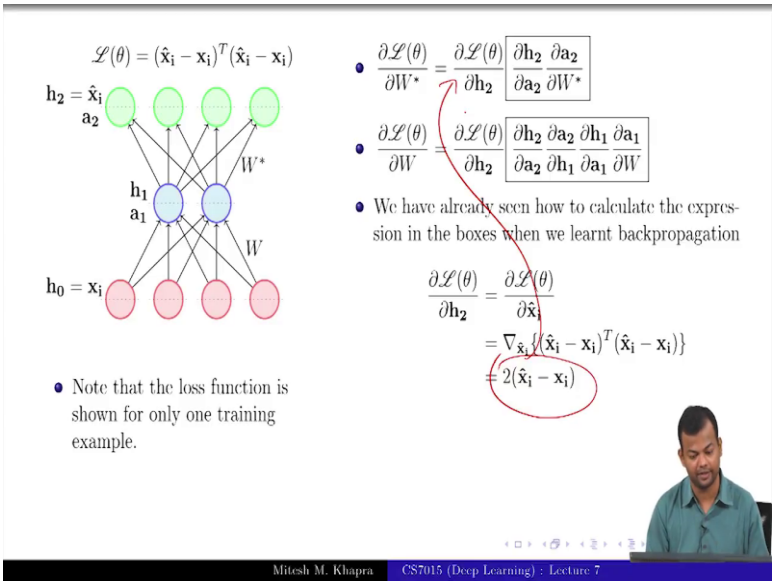
Now, this portion as I was trying to say is something that we have already seen in the back propagation lecture, and nothing has changed in the network in that part. So, you can just reuse it as it is and this portion is something that we need to recomputed right that is the only thing that we need to recomputed and plug it into our back propagation code or the algorithm, which we had in the previous lecture. And similarly if you want to do dou l theta by dou W it is the same idea here that you could write it as the following chain. And this part of the chain you already know how to compute from the back propagation lecture.

All you need to do is change the loss function and just try to find the derivative of the loss function with respect to your output layer which is h 2. That is the final thing that you have changed just as in my toy example I had changed e raise to y 3 to square root of y 3 right? That is the similar change that I am trying to do here fine.

So, all we need do is dou l theta by dou h 2, but dou h 2 is the same as x i hat right because that is my output and my output I am calling it as x i hat. So, I need to take actually the derivative of this. So, I am just using the vector form here I could have also written it as this summation over i equal to 1 to n x i j minus x hat i j the 2 whole square right? I could have also written it as am I just writing it as the vector here in the vector form here right, but this quantity ultimately is going to be a scalar because it is a dot product between 2 vectors which is the scalar.

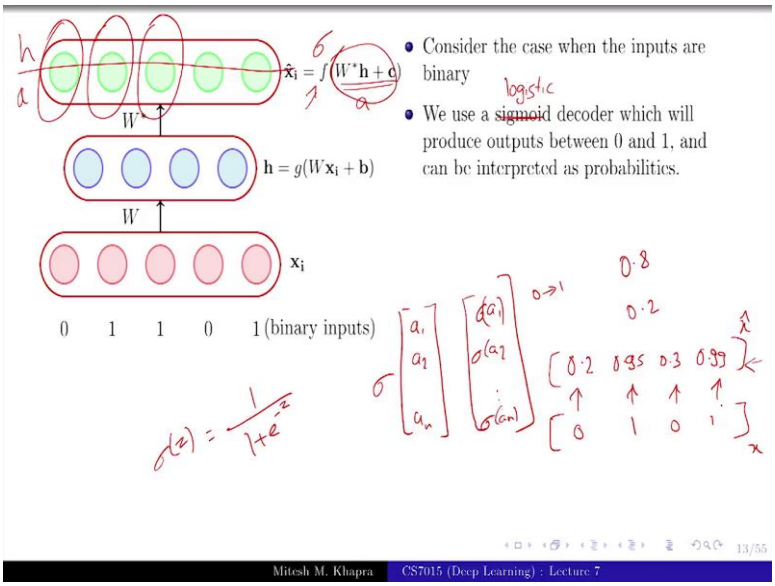
So, what I am doing here is taking the derivative of a scalar with respect to this vector. So, what is that derivative going to be? It is going to be a vector.

(Refer Slide Time: 27:20)



And I am just, so we have similar stuff in the past. So, you can actually easily work this out. So, this will actually turn out to be the following vector which is to times x i hat minus x i right? So, this is very simple I have just computed this and all I need to do is go back and change my back propagation code. And change this derivative of the loss function with respect to the output clear and the rest of the code I can just reuse it as it is. So, now similarly, so we have both of these ready.

(Refer Slide Time: 27:50)



Now, let us look at the other case when we have binary inputs ok. This is the most more this is something different that we will have to do here. So, we will now look at the second case where the inputs are binary. So, first we look at case when the inputs were real numbers, and hence your outputs also needed to be real numbers.

Now, we look at the case where inputs are binary and hence your outputs also need to be binary ok? Now, here, so each of these guys is actually a sigmoid functions. So, it is in or rather if you look at the output you could divide into two parts. So, this is the pre activation and this is the activation. So, your this is actually the pre activation and this is the activation right?

So, this activation is actually chosen as the sigmoid function or the actually the logistic function not the sigmoid function of course, logistic is the sigmoid function, but the logistic function which was 1 over 1 plus e raise to minus z right? So, logistic of z is equal to 1 over 1 plus e raise to minus z. And remember that this sigmoid function was element wise.

That means, this is a is a vector it has elements a 1 a 2 up to a n and then you know apply the sigmoid to it you get h, which is going to be sigmoid of a 1 sigmoid of a 2 and sigmoid of a n right? So, it is just the sigmoid applied to every element of the activation layer. That means, every element of this vector which have circled.

So, now in this case your outputs are going to be between 0 to 1 right, because your inputs were also between 0 to 1 and your sigmoid or the logistic function is going to give you clamped outputs between 0 to 1. So, since this is between 0 to 1 we could actually interpret it as probabilities right? So, we could say that whatever you are reconstructing is actually telling you that with 0.8.

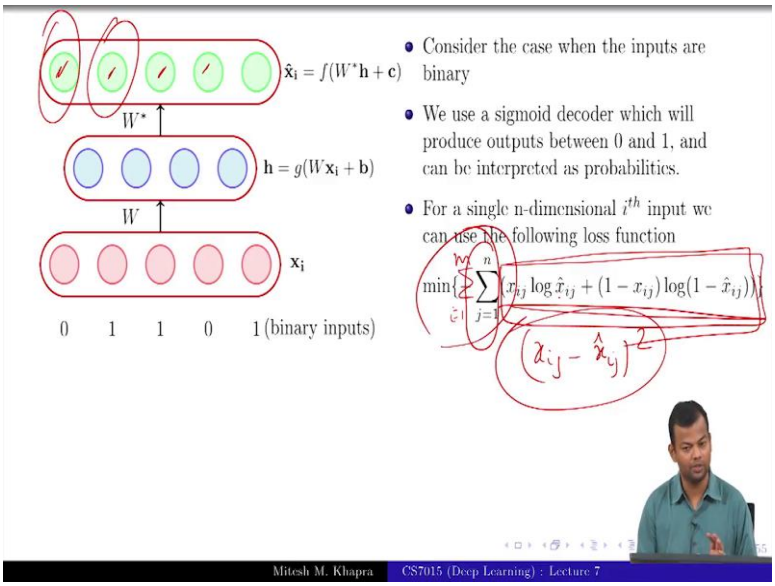
Suppose, the reconstruction value is 0.8, then you could think of it that with probability 0.8 it is telling you that the output should have been 1 right? And if it tells you that the output is 0.2 if the sigmoid gives an output as 0.2. Then you could think of it that with probably 0.2 the output was actually 0 or rather the input was 0 because an input is the same as the output.

So, that is one way of interpreting it and this way of interpreting it why does it make sense. So, we will just look at that right, so before at if I do not give you this interpretation. And remember that the sigmoid is going to produce values between 0 to 1, but not necessarily 0 and 1 right it will try to be as close to 0 when the input is 0, but it could also produce 0.05 and so on. And when the input is point nine it could also produce something like 0.95.

So, at the output also you are going to get these vectors which are of which would look something like this right? And suppose you are input was 0 1 0 1 now can think of a suitable loss function for this yeah. So, again these are two vectors these are x hat and x. So, once again you could have just gone with the with a squared error loss right, you could have taken the squared error difference between these two and you could have been fine.

So, that is definitely one way of going about it, but whenever we are looking at these binary inputs. And whenever this probabilistic interpretation is possible we tend to do something better which is look at the cross entropy loss instead of looking at the squared error loss. So, I am not saying that the square error loss is wrong in this case, but you could also use this cross entropy loss. And in practice for our binary inputs the cross entropy loss often works much better than squared error loss.

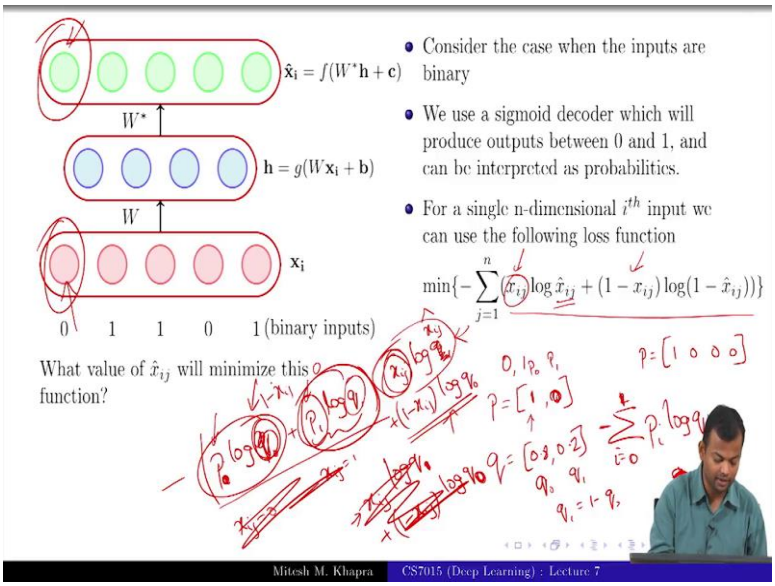
(Refer Slide Time: 31:31)



So, let us see what I mean by the cross entropy loss. So, remember that you have n outputs right? That is why this summation let us not worry too much about what is written inside for the time being I will explain that, but that is the I just want to explain the summation first. So, what you are saying is that for each of these green guys at the output you are going to make some loss. And you just want to some over that loss that is what we are trying to see.

Now, ideally you could have just written it as just done what you had done before and written this entire replace this entire box by this squared error loss. And that would have been just fine right of course, there should have also have been this summation i equal to 1 to m here, because you are going over all the m training instances and for each of the m training instances you are trying to minimize this loss. So, this 2 summations followed by this squared error loss would just have been fine.

(Refer Slide Time: 32:26)



, but instead of that I have this something special here ok. So, let us look at what this special quantity is, ok? And now for that remember that I am trying to interpret each of these inputs as a binary random variable. I am saying that they can take values 0 or 1.

So, I can think of it that when I am given that this value is 0. I can write it as this deterministic probability distribution where I have p. And the probability mass is entirely concentrated out on this 0 value and my the probability mass on the value 1 is 0. This is something similar to what we had done earlier when we are given these labels suppose it was apple, orange, mango and banana. And the class label was given to us that this is an apple. Then we could still write it as the probability distribution where all the mass was concentrated on apple and everything else was here.

So, I am saying something similar here right? So, you could think of it that two possible values can occur here 1 and 0 and if I tell you this is 0, right? Then I am telling you that with probability 1 into it is 0 and with probability 0 it is 0. So, I still write it as a probability distribution now the same thing I can have at the output. So, for this unit when I am trying to reconstruct it and if I produce the output as 0.2 then I can or rather let us say 0.8 then I can say that with 0.8 probability I am predicting 0 and with 0.2 I am predicting a 1 right?

So, now I can think of this again as two probability distributions. And once I some have two probability distributions I know that cross entropy is the right or a better loss

function to look at right? And what is cross entropy actually in this case it would be given by summation i equal to 1 to 2 right or rather i equal to 0 to 1 because if the those are the values it can take p of i right into log of q i plus yeah. So, p of i into log of q i that is how I can write it.

So, let me just since there are only two terms I can just expand this summation right? So, I can write it as p i or rather p 0 log of q 0 plus of course, it is a minus sign here, this is a minus sign at the out p 1 log of q 1. I can just open up because there are only two terms. So, I can write it as this is that fine ok?

Now, also I know that there is this relation between p 0 and p 1 right? That p 0 is actually 1 minus p 1 yeah. Similarly, you have this relation between q 0 and q 1 that q 1 is equal to 1 minus q 0 because the sum is going to be 1ok now let us look at this sum right? So, in the binary case this sum becomes interesting because. Now suppose your input x i j right which is the entity that I am looking at, suppose that was equal to 0. In which case all the probability mass would be concentrated on p 0 and p 1 would actually be equal to 0, which means the second term would display.

On the other hand if x i j is equal to 1 then the reverse situation what happen, that everything would be concentrated on p 1. That means, p 1 is equal to 1 and this guy would become 0 because p 0 is going to be 0 right? ok So, there is this another way of writing it that you could day that instead of x instead of writing p 0 and p 1 you could just write it as x i j right into log q 0 plus 1 minus x i j into log of q 1.

So, now let us look at it again, so when x i j is 0 first which is the same which happened here just an (Refer Time: 36:20) in same thing right? Because, whenever s x i j is 0 p 0 is equal to sorry it should have been q 1 and log q 0 sorry i made a mistake here. So, it have been x i j into log q. So, or rather let me just rewrite it.

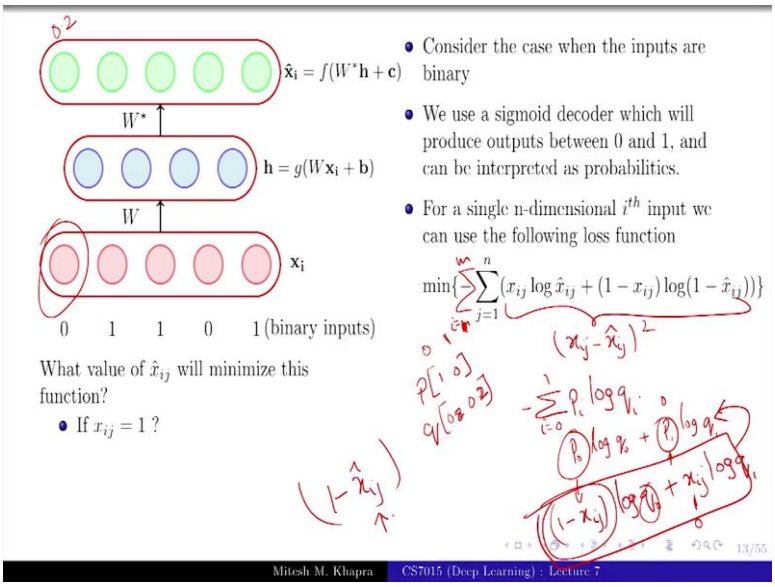
So, this is going to be actually I can write it as, I look at this term first. So, I can write it as x i j into log q 1 and then the second term I am going to write it as 1 minus x i j into log of q 0 right? And then I am going to simplify this further, but let see what is the consequence of this.

So, now whenever x i j is equal to 1 this term will remain and the second term will disappear and that is exactly what was happening in our original formula right? So, this

is just an equivalent way of writing your x i j is equal to 0 this term will disappear, but this term will remain; that means, log q 0 will remain this is exactly what was happening in our original formula right?

So, that is so now, I have given you why a I can replace p 0 and p 1 or rather p 1 and p 0 by x i j and 1 minus x i j. And now I can make a similar argument for x hat i j also. So, I can think of q 0 as whatever s predicted at the output right sorry I can treat q 1 as whatever is predicted out 1 output. So, whatever my sigmoid function predicts I can think of it as it is predicting the probability of getting a 1 right? So, it is just predicting the heads probability or the probability of getting 1. So, I can instead q i q 1 i can write it as x i j hat, and similarly instead of q 0 I can write as 1 minus x i hat i g right? So, did you get that so these become very messy.

(Refer Slide Time: 38:13)



So, let me just clean this up and I will just go over this again right?

So, what I was trying to tell you is that in the ideal case you could have just replaced this by the squared error loss, but since you are dealing with binary inputs you can do something better because you can interpret the outputs as probabilities. So, when you get a 0.2 here you can interpret it as it is telling you that the probability of this unit being 1 is 0.2, it is very less. And that is the same as saying that the probability of this unit being 0 is 1 right?

So, you can interpret this as a probability. Now if you think of it that way then you can say that at the input you are actually given a probability distribution. So, which tells you that in the first case your probability distribution looks like 1 0 right, because all the mass is focused on value 0 because your input is 0 at that case. And now suppose your output was 0.2 right and 0.2 is what you are treating as a probability of. So, this is the probability of 1 this is the probably of sorry this is the probability of 0 oops and this is the probability of 1.

So, if your output is predicting 0.2. That means, it is predicting 0.8 for 0 and 0.2 for 1. Now if you think of it this way then you can capture the loss function between these two guys using the cross entropy formula. Which is going to be summation i equal to 0 to 1 p i log q i is that fine? ok And now I just said that the since there are only two terms I can just write it as p 0 log q 0 plus p 1 log q 1.

Then I focused on this relation between p 0 p 1 and your input. So, whenever your input is 0 ok your p 0 is going to be 1. So, then I can just replace p 0 by 1 minus my input right? So, if the input is 0 then this guy is going to be 1 and that is exactly what this expression is also going to be.

So, I can write it as 1 minus x j log q 0 and similarly for this second guy. Whenever input is 1 this guy is going to be 1, whenever my input is whenever my input is 1 this p 1 is going to be 1 whenever my input is 0 this p 1 is going to be 0. So, I can just replace p 1 by log by x i j and now you can see that this expression evaluates to the same as this expression right, you can substitute value of x i j 0 or 1 you will get the corresponding p 0 p 1 which would be 1 or 0 depending on what your input was and these two expressions will evaluate to the same thing.

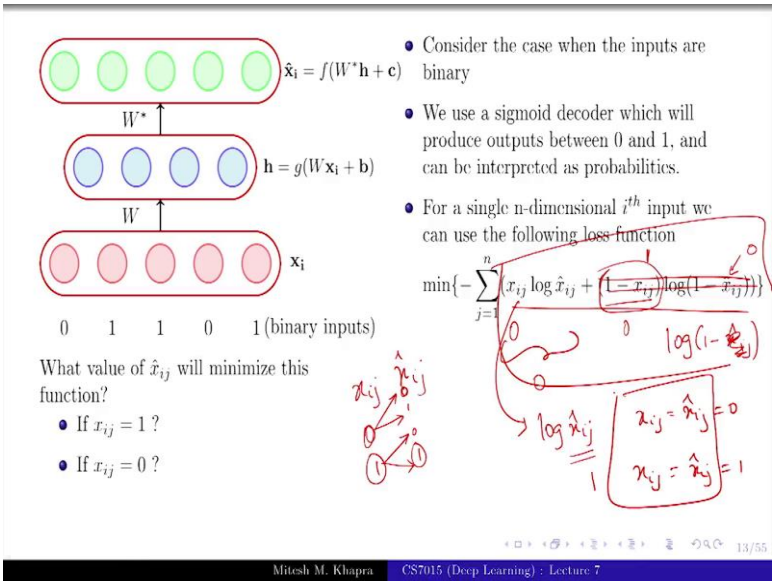
So, just as I replaced the p's by x i's x i j's, I can similarly replace by a the q's by x i j hats right because once again q 0 is nothing, but 1 minus whatever my output was predicted because whatever is predicted I am treating as the probability of getting a 1. So, 1 minus that is going to be the probability of getting a 0. So, that is what q 0 is and similarly q 1 I can replace by x hat i j. And so that is exactly what I have done in this expression here.

So, now this expression every term in these n terms captures the cross entropy for that particular random variable right? So, this is the original distribution p for this random

variable. This is the predicted distribution q for this random variable, and I have just told you that this the cross entropy between these two distribution can be written in this simple form as the function of x i j and x hat i j.

So, this is the standard thing to do when you are dealing with Bernoulli random variables. So, you can go back and read up a bit about it ah, but for now I guess with this explanation it should suffice to know why this expression is used. And remember that I am not telling you that this squared error function was bad. I am just telling you that instead of the squared error function cross entropy loss function works better when you are dealing with binary inputs.

(Refer Slide Time: 42:34)



So, with that let us pursuit and the another we have looking at it is the following you can now look at this expression. And tell me when is this expression going to be minimized. So, we have x i j and x hat i j you can see that this expression will be minimized only when x i j or rather x hat i j is equal to x i j right? So, now, x i j could take value 0 or 1 ok and now x hat i j could take 0 1 or 0 1.

So, you can see that for these two combinations the value is going to be minimized only when x hat i j is actually equal to x i j. That means, if x hat if x i j was 0 then x hat i j should also be 0. And similarly in this case also if x i j was 1. Then the expression will be minimized only when x hat i j is equal to 1. So, let us see this so suppose x i j was 0.

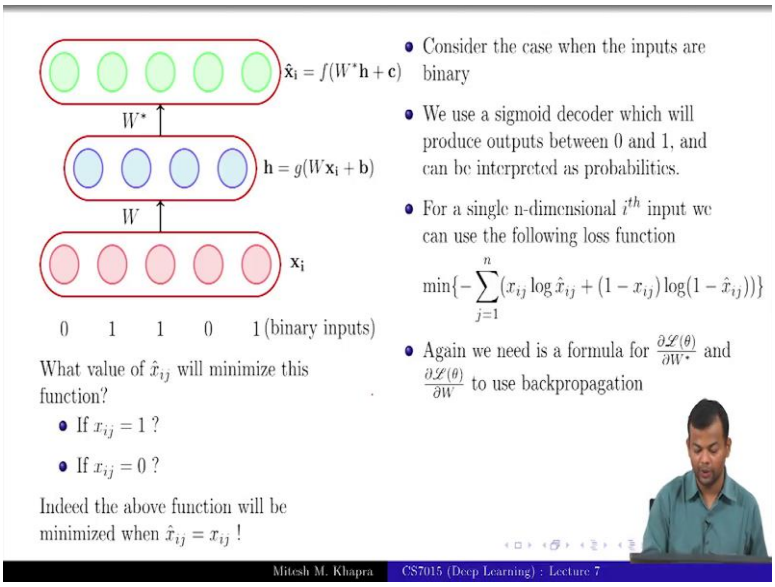
That means, this term is going to go to 0, but this term is going to remain and now if you are x hat i j was not equal to 0.

Then you will get some log of 1 minus x hat i j as the loss right, but if x hat i j was also 0 then you would get log of 1 which is 0. So, this whole expression would then evaluate to 0 which is the minimum possible value for this expression right?

So; that means, if x i j is 0 then this expression will be minimized only when x hat i j is also equal to 0. Similarly, if x hat i j sorry if x i j is 1 then this 1 minus 1 will give you 0. So, this term is going to disappear, but this term will remain. So, this will just be log of x hat i j because x i j is equal to 1.

Now, if x hat i j is also equal to 1 then this is become log of 1 which is 0. That means, again this expression will attain it is minimum value when x hat i j is equal to x i j is equal to 1 right? So, this expression now attain it is minimum value in two cases when x i j is equal to x hat i j is equal to 0 or when x i j is equal to x hat i j is equal to 1. So, compactly I can say that this expression will attain it is minimum when x i j is equal to x hat i j, that is why this loss function makes sense.

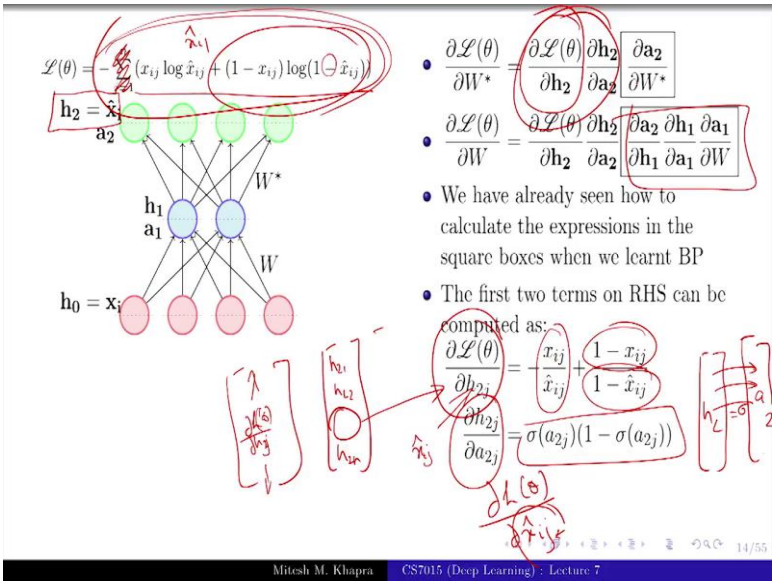
(Refer Slide Time: 44:54)



Now, again we have this problem that we want to use back propagation to train this network. And once again for back propagation we will need the following gradients the gradients of the loss function with respect to W and W star ok. This is what we are going

to need and I am going to make this same argument again that whatever hard work you had done in the back propagation lecture you can just reuse all of it.

(Refer Slide Time: 45:19)



Because the only thing your changing is this final loss function. So, you just need to compute the gradients with respect to this loss function and everything else is going to remain the same right?

So, that is exactly what I am going to do on this slide. So, whatever is in the boxes here these two boxes that is something that you have already computed. And now what I am going to compute is the stuff which is outside the boxes, so let us look at that. So, I am interested in computing this dou l theta by dou h 2 this is the derivative of a scalar quantity with respect to a vector, say it is going to be a vector. And I am going to follow our usual recipe which is h 2 is actually equal to h 2 1 h 2 2 up to h 2 n.

So, I am going to consider any of these guys which is h 2 j, I am going to compute the derivative of the loss function with respect to this one entry and since I have that I am going to construct the entire gradient right? So, now, I will have this dou l theta by dou h 2 j right and once I have that expression I am just going to generalize it to all the other entries in this vector.

So, let us look at that expression first ok So, now, if you look at this actually it does not have an h 2 j right, but we know h 2 j is the same as x hat j or rather x hat i j right for the

ith input it is going to be x hat i j. Because h 2 is equal to x hat i ok, you can just see that the top left corner of the slides say x 2 is equal to x hat i. So, this is nothing dou l theta by dou x hat i j.

So, now I want to take the derivative of this quantity with respect to 1 particular x i j, and remember that this quantity has the sum which is indexed over j. So, j goes from 1 to n I am looking at 1 particular j. So that means, if I expand this sum of all the js possible the derivative with respect to all, but 1 is going to be 0, because they do not depend on this particular j. So, if I am looking at j equal to 3 then the term which has x hat i 1 is going to the derivative of that term is going to be 0.

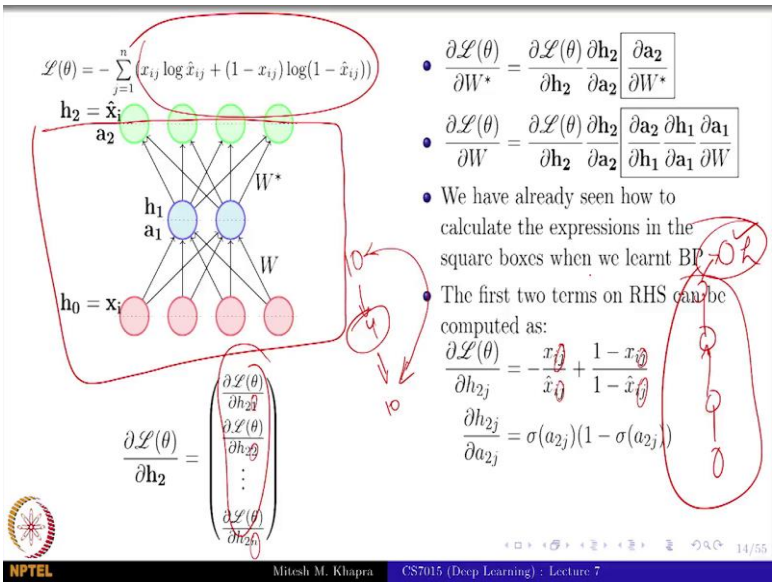
So, for all these terms in the expression only that term where a j is equal to the j which I am considering is going to remain ok. So that means, only one term in the summation would remain and for that one term, so let me just rid of the summation right? So that means, only one term in the summation would remain, I am trying to find the derivative of this quality x which has a lot x i j's with respect to x i j.

So, now this is of the form a log x, so the derivative would a over x right? So, that is exactly what I have written here and similarly for the second guy this is 1 minus a into log of 1 minus x. So, the derivative is going to be 1 minus a over 1 minus x and of course, there is this minus sign here which will then get adjusted appropriately right? So, that is how this expression has been completely that is very straight forward and now as you need the derivative of h 2 j with respect to a 2 j.

So, remember that h 2 is equal to sigmoid of a 2 which means it is just an element wise sigmoid right? So, I just need to compute the derivative of the j'th entry of h 2 with respect to the j'th entry of a 2 all the other derivatives are going to be 0 because they do not depend on that particular entry of a 2. So, now, that is just going to be sigmoid of a 2 into 1 minus sigmoid of a 2 right?

So, I have computed these two quantities I can just plug it then back into back propagation code. The rest of the code is going to remain the same and I have the gradients ready with me.

(Refer Slide Time: 49:14)



And as I said once I have this one guy I can just extend it I can just generalize it. So, I just had these j's here right for h 2 j, so I can just replace the j by 1 2 up to n and I will get the same expression.

So, that is the end of module 1 where we introduced auto encoders. What we showed is that they are actually just like any other feed forward neural network accept that they have this special objective. That they want to reconstruct the input and the reason they want to reconstruct the input is they about to first create a bottle neck which is this h hidden representation. And then try to reconstruct from there and just as I gave you that compression analogy that you have this 10 bits you want to compress it to 4 bits and then reconstruct the entire input again.

So, this will happen only if these 4 bits capture everything that is required or the most important characteristics of your original input right? And then we could have a loss function which tries to capture the difference between my original input and my reconstructed input.

Now, we argued that this loss function will be dependent on the nature of your input. So, for the real inputs it was straight forward we just said that we can use the squared error loss function for the binary inputs we actually did something special. We said that we can actually use the cross entropy, and then we had this funny way of writing the cross entropy which was this x i into log of x hat and 1 minus x i into log of 1 minus x hat.

And just gave you some intuition that that is the same as writing p log a pi log or rather p 0 log q 0 plus p 1 log q 1 write and the I just gave you some explanation for doing that.

You can go back and check on how do you write the cross entropy for Bernoulli random variables and you will see that this expression makes sense. And once we had this expression computing the gradients was easy. So, the other thing that we relied on is that in the back propagation lecture we had taken care of everything up to this point and in this lecture we have actually changed the loss function.

So, 1 loss function was the sum of squared, squared loss errors and the other loss function was the sum of sum of cross entropies whereas, in the back propagation lecture we had only dealt with cross entropy by the case that we made is that sense you have this chain. All you have done is change the last function in the chain right, you have changed this l function all the other functions you have not changed.

You can just reuse the computations from these or you can just use the code that you had written for these in the back propagation assignment. And you just need to change this last guy to adjust for the change in the output layer or the change in the loss layer. So, with that we will end the introduction to auto encoders, there we have done we have actually covered how to train an auto encoder using back propagation.

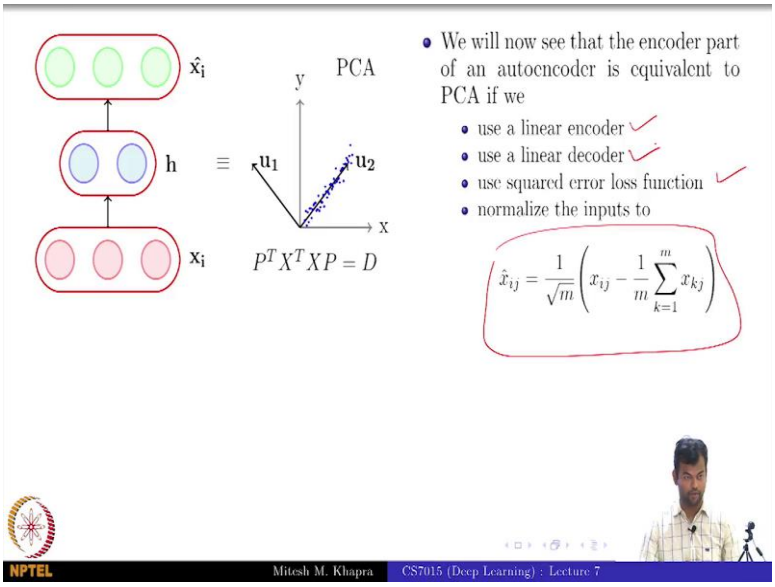
Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 7.2
Lecture – 07

Link between PCA and Auto encoders

So, we will move to the next module where I would like to show you a link between PCA and Auto encoders.

(Refer Slide Time: 00:20)



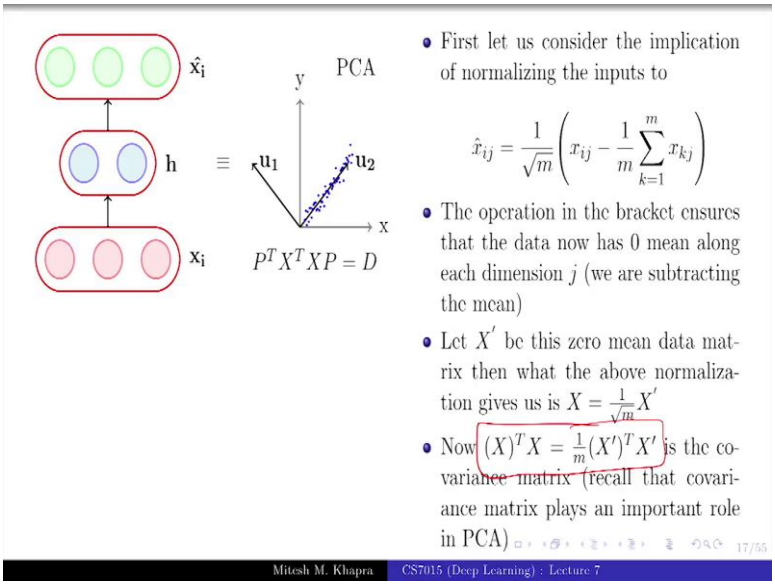
So, this is what I am trying to show you that under certain conditions, PCA is or rather an auto encoder is equivalent to a PCA. And the conditions are; if you use a linear encoder, if you use a linear decoder, if you use a squared error loss function and if you normalize the inputs to this. So, for the time being, just ignore the last bullet. Let us look at the other 3 bullets using squared error loss functions.

So, remember I gave you different choices right? You could have used the cross entropy or the squared error loss. But I am going to prove this equivalence only under the condition when we have the squared error loss. What do I mean the u encoder is a linear encoder? g is a linear function. We are not using a sigmoid or any logistic or anything like that and linear decoder. Again the same thing; we are not using the sigmoid or soft

max or anything at the output; it is a linear function. Under these conditions, I will show that or I will try to show you that PCAs equal auto encoders equal to PCA.

What does this mean actually? Now what do I mean by it is equivalent? What do I have to show you actually? How many of you understand what I am trying to prove? How many of you can mathematically define it? Ok, so, we will try to make this clear over the next 15 minutes.

(Refer Slide Time: 01:43)



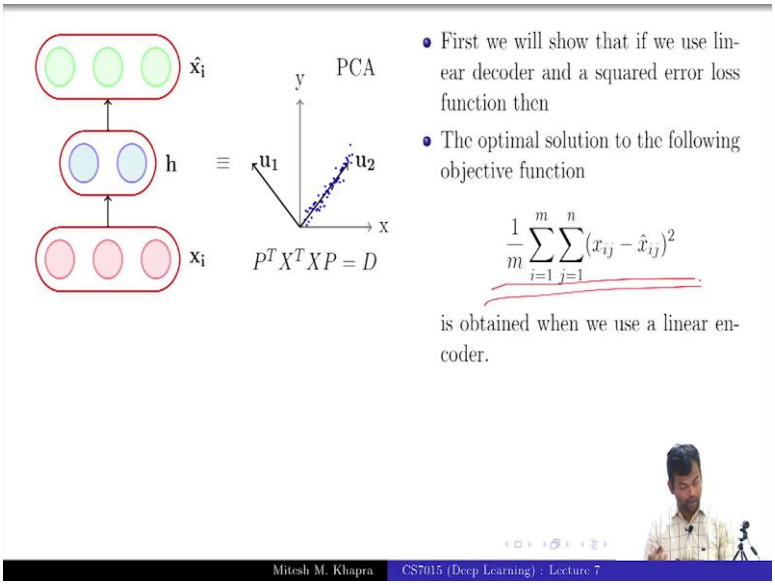
First, let us look at the last condition right; which I ignored ok I always anticipate all this right? So, I have full faith in you guys ok. What is this mean? Now, what I am doing? Centering the data and I am also doing 1 by square root of m; why? Mean, as the standard deviation.

So, the operation in the bracket ensures that your data now has become 0 centered right? It is a 0 mean. And now let X dash be this matrix this one right such that, all it is elements are 0, mean is this still a flicker again alright.

So, let I am calling X dash as this matrix ok. So, this matrix, where I also have 1 by square root of m, I can write it as everyone gets this is simple. Now do you see where this is headed? What would X transpose X be? Covariance matrix; So, I needed that 1 by m right at the out.

So, now this is the co-variance matrix. So, if I do this normalization to the original data and then if I take let x dash be that quantity and then if I take x transpose x then I will get the co variance matrix everyone gets this that I did this to get the co variance matrix. So, that I mean I did this. So, that when I take x transpose x I get the co variance matrix after this normalization only it will be the covariance matrix.

(Refer Slide Time: 03:04)



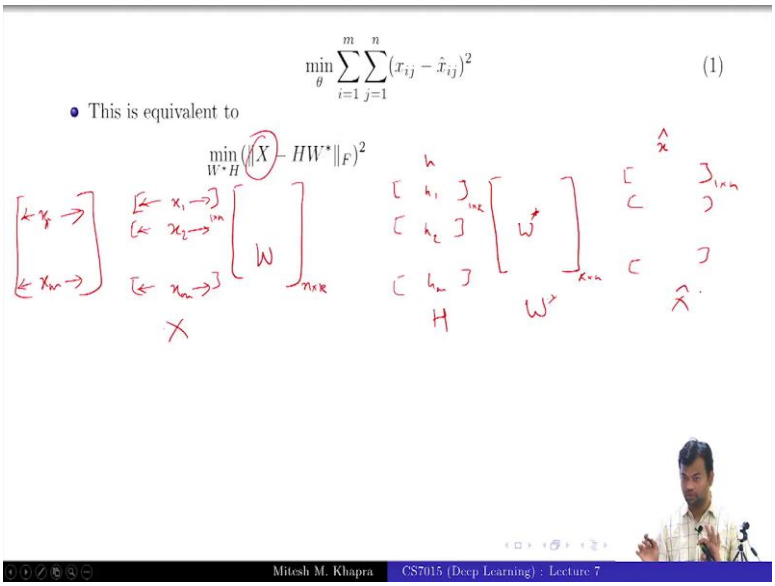
So, first we will show that, if we use the linear encoder, decoder and a squared error loss function, then the optimal solution to the following objective function. What does this objective function?

Student: Squared error.

Squared error loss is obtained when we use a linear encoder. Do you understand the implication of this? What does being stated here? Ok, So, I have fixed the decoder. I have said that the decoder is going to be a decoder. I have fixed the encoder or I have fixed the loss function. This is going to be a squared error loss function. This is given to me. Now under these conditions, I am trying to minimize this loss function ok.

Then I am telling you that the only solution to this is that the function dash should be a linear function which function? The function g should be a linear function you cannot choose sigmoid or logistic or anything else right? The optimal solution will occur when g is a linear function everyone gets what is being stated here.

(Refer Slide Time: 04:02)



So, this summation that I have written right or. In fact, this the entire objective that I have written is actually equivalent to this objective. Is this fine with everyone? Even though I have not defined what H is just fine with everyone. So, we had X which was X 1 to X m ok. I had picked one of these Xi's. What is the dimension of this?

Student: 1 cross.

1 cross m and then I had multiplied it by a weight matrix W. Not W star; remember that what do the dimension of W.

Student: n n.

N cross k and what will I get as the output.

Student: (Refer Time: 04:58).

I got an H which was 1 cross k, what did I do this?

Student: Multiply it by

Multiply it by.

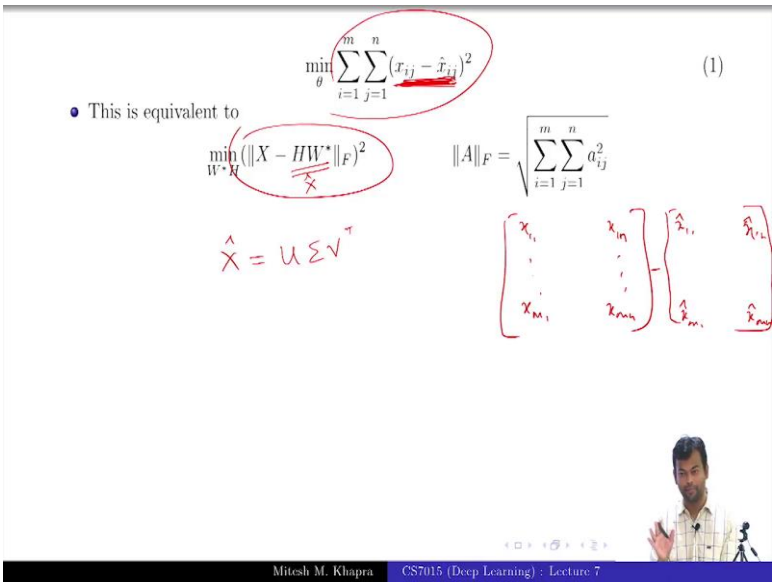
Student: W star.

W star which was k cross m and what did I get as the output?

Student: x hat.

x hat which was 1 cross n right? So, what I am telling you is that, I could do this together for all these X i's. I could do this operation at one go and I can call this as X matrix and what will I get here? h 1 to h 2 to h m. And I can call it as the H matrix and I multiply it by W star and what do I get? X cap ok. Is that fine? ok, but without defining these things also it was fine. So, it does not matter.

(Refer Slide Time: 05:51)



So, now how many of you get that this quantity is the same as this quantity? Now how do I explain this was the Frobenius norm of a matrix? Some of the squares of the elements.

Now, what is the matrix X? It is the X 1 1 up to X 1 n and X m 1 up to X m n and all elements in between right? What is the matrix H W star; we just did that the same thing expect that it is X hat.

Student: (Refer Time: 06:04).

I take the difference between these 2 what do I get? Every element of that matrix is equal to this quantity that I have underlined right? So, I get a new matrix such that every element of that matrix is equal to this quantity. Is that fine? Now, I am taking the square of every element of that matrix and adding them up what is that equal to?

Student: (Refer Time: 06:15).

A Frobenius norm how many of you get that now? Almost everyone, ok. So, this is equivalent to the Frobenius norm ok. Now, where have you seen the Frobenius norm before what did we show in the SVD theorem?

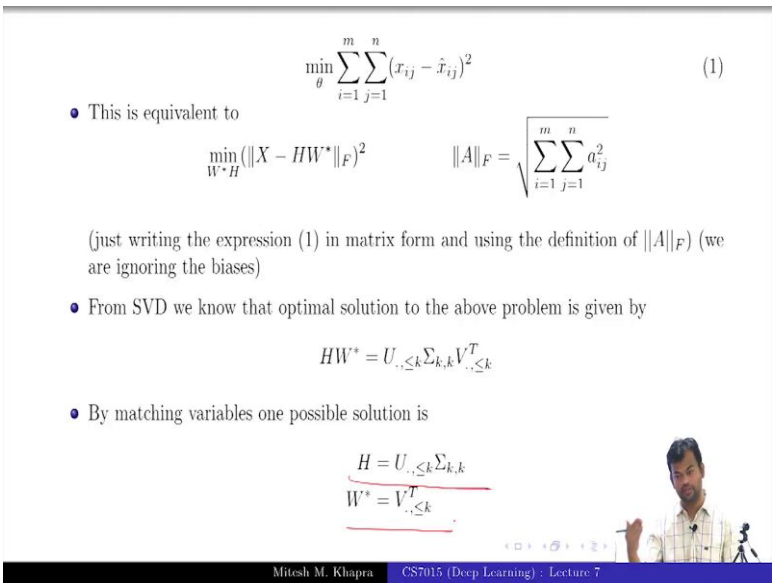
Let us try to connect things right if you do not learn how to connect things it is going to be very difficult. What is this X hat? It is a dash of X.

Student: Reconstruction.

Reconstruction it is a dash of X approximation. What is the solution to this optimization problem? What is the solution to this optimization problem? I shall started off with the answer that we saw this in the SVD theorem and then I asked you a question what 30 hours 32 hours; not even 32 hours are passed since we did this. Come on, what is the solution to this? No, no that is fine.

But what is the solution X hat is equal to what? The best approximation to X is given by what? Is it fine yeah. So, some k yeah, but it is going to come from the SVD theorem, right is that fine? It depends on what rank approximation you want, but it the best approximation to this is going to be given by the SVD of X, is it ok? Everyone gets that yes forgot about it, but now do you remember it all those extra lectures 8'O clock in the morning.

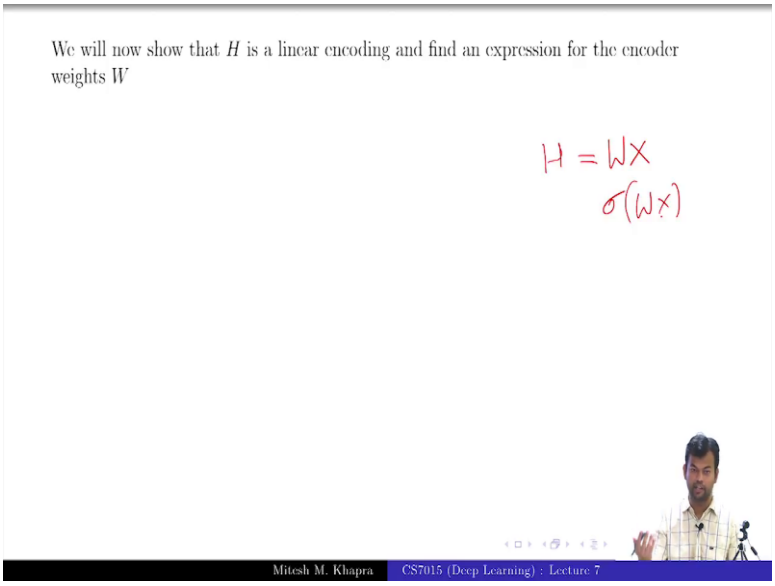
(Refer Slide Time: 08:26)



So, that means, H W star should be equivalent to this that we know from the SVD theorem that, the optimal solution is going to be given by SVD. So, if I just compare terms ok, then I could write that one solution is this that H. H is equal to U into sigma and W star is equal to V transpose. I could have chosen the other solution also where H is equal to V or sorry U and W star is equal to sigma V, ok. But I will work with this particular solution. You see, this I am just matching variables right? It is said that, A B is equal to C D E. So, I am saying that A is equal to C D and B is equal to E.

Now, we will work with this. So, and we will try to show something; so, let us see what we are trying to show.

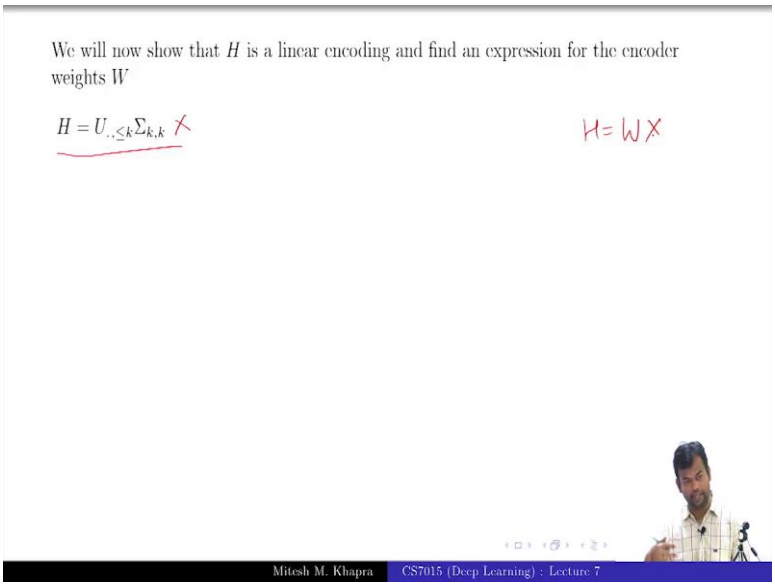
(Refer Slide Time: 09:14)



Now, first thing that we will show is that H is actually a linear encoding. So, what does this mean? You first always understand what has been tried to prove right? I am saying that, I am going to show that H is a linear encoding of X, then what is it that I am trying to show?

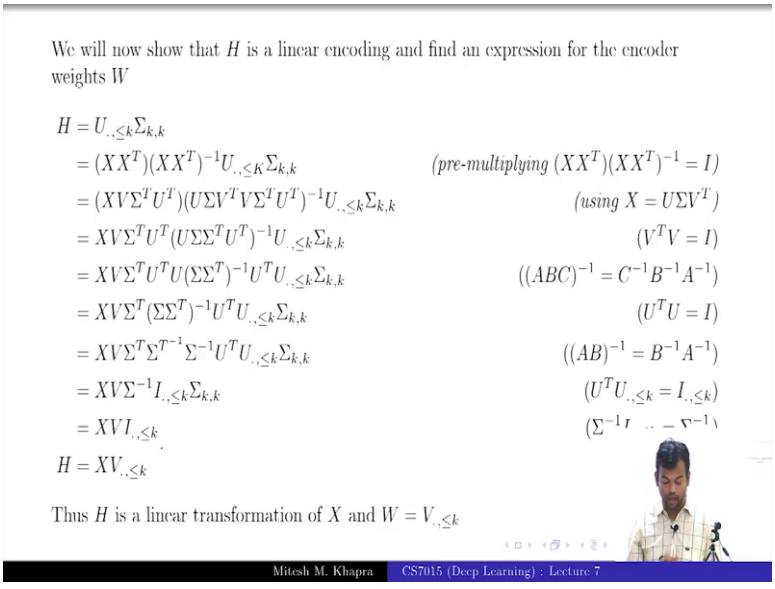
I am trying to show that H is equal to a linear encoding of X when H is of the form W X and not something of the form W sigmoid of W X or something like that or any other non-linearity for that matter. Is the statement clear? That is what I am trying to show. When I say H is a linear encoding, I mean that H is obtained by a linear transformation of X.

(Refer Slide Time: 09:52)



Now, H as we defined on the previous slide is equal to this. Now, if I already had an X here, then I was done, but I do not have any X there yet. So, I want to a get to a form where I can show that H is equal to W in to X. So, I will just do some simple trickery and arrive try to do arrive at that form.

(Refer Slide Time: 10:13)



So, the first thing I am going to do is pre multiplying pre multiply by this quantity and this is fair because this is just equal to I what next I will write these 3 x's as u sigma v transpose and I will leave one x as it is that.

Now, just can you just try to see what the next step would be this V transpose V will disappear because it is equal to i. Now what happened here? I actually just expanded this inverse. So, I will think of this as A B C. So, A B C inverse is equal to C inverse B inverse A inverse.

So, I have just applied that it just that my inverse is a very straight forward matrices here they are just the transform of the original matrices. Everyone gets this step. Well you can stare at for a for a few more seconds if you want. How many of you do not get this? How many of you get this? Ok, now what is next this U transpose? U disappears.

Student: (Refer Time: 10:59).

This also disappears.

Student: (Refer Time: 11:03).

No.

Student: (Refer Time: 11:06).

It is this U is only. The first k columns of U right, this is not the entire U. This is just the first k columns of U, fine. Now what next A into B inverse is?

Student: B inverse.

B inverse A inverse what will happen? Now that quantity will disappear. So, what do you have left now ok. So, this is something ok. So, now, let us look at this is let us say this is n cross n and this is n cross k what is the output going to be.

Student: n cross k.

N cross k and what is the output going to look like is the first k columns of.

Student: Identity.

The identity matrix everyone gets that if you do not you can just work it out with the small matrix after going home and you will get it right if; so if I done the full multiplication, I would have got the identity matrix. But I am just talking the first k

columns. So, I will get the first k columns of the identity matrix. Do not fed too much. If you are not getting this, you can just work it out on paper and you will get it.

So, I get the first k columns of the identity matrix and this inverse disappears this sigma transpose into sigma transpose (Refer Time: 12:19) now what next what is this product going to be the first k elements of.

Student: Sigma inverse.

Sigma inverse and that is going to get multiplied by sigma k cross k. So, that will give me the first k elements of.

Student: Identity.

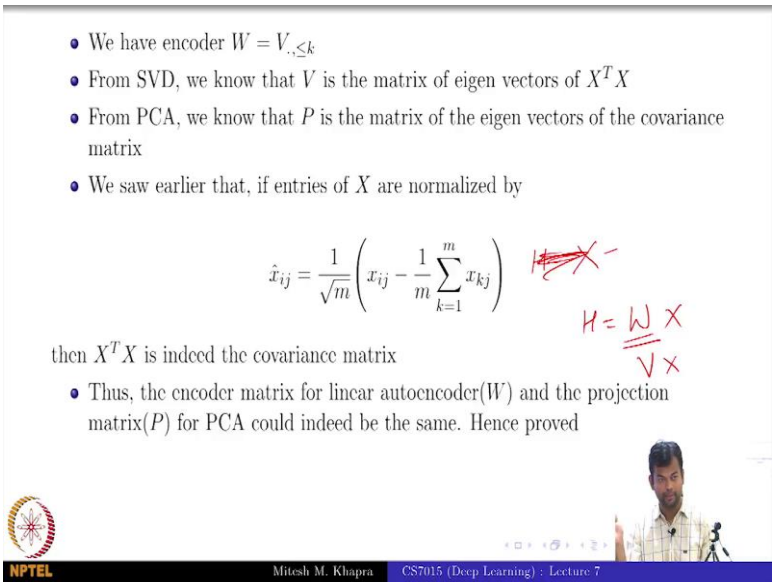
Identity matrix there is some very simple matrix operations where you are just taking some columns right? So, if you do not understand this right? Now do not worry. You can work it out. Everyone is confident, they can do this, please raise your hands if you are confident. And now, what do I finally, get this multiplication will give me.

Student: The first k columns.

The first k columns of V ok; so, have we come to the desired form what I have shown. Now H is a dash of X or linear transformation of X; that means, my optimal encoder was a linear encoder and what was the optimal weight matrix w the first k columns of V yeah I someone pointed it last time also I could not. I ignored it. I will just pretend I understood.

But I get it I know that there is a simpler solution. I do not know why do it this way, but there is a simpler solution. I just like making life miserable for you guys, but, but the point is, you can figure it out, that it is a it is a linear transformation of X now.

(Refer Slide Time: 14:00)



We have that the encoder is equal to the first k columns of V ok. What is V eigenvectors of X transpose X ok?

Student: A.

What is the other thing that you know about the eigenvectors of X transpose X they are the solution for the.

Student: Eigen.

If you have given an matrix X then the PCA is the eigenvectors of the co variance matrix was the co variance matrix X transpose X what is are it is eigenvectors capital V right? So, what have we arrived at are we done with the proof. Yes, how many of you think that done with the proof? How many of you think that we are done now?

So, it is done right? So, we have proved what we wanted to prove right? So, what did we want to prove that you are doing auto encoders. You are trying to train an auto encoders and you are loss function is the squared error loss function. We saw a neat way of writing that squared error loss function as a matrix operation where X minus capital H into W.

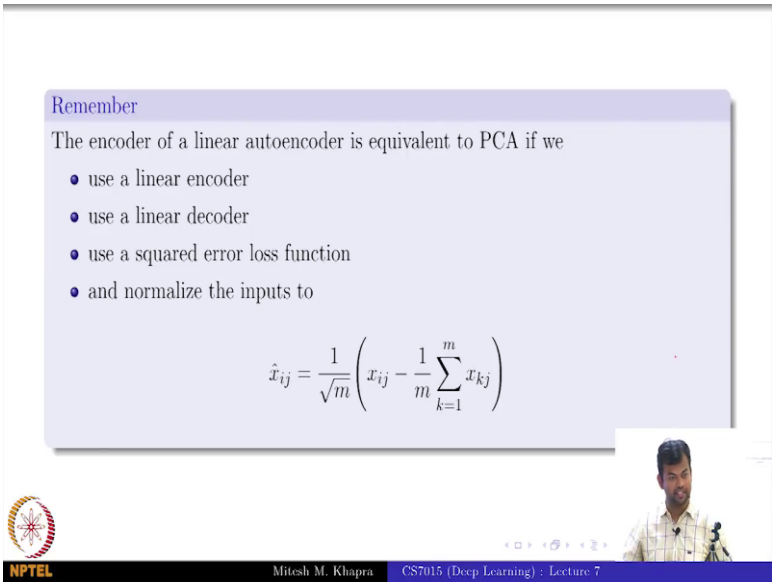
And then, we saw that these squared error loss function is nothing, but the Frobenius norm of this and we knew that the minima of this objective function the Frobenius norm

of X minus H W would occur when S W is equal to SVD of X right? We started from there and showed that H is actually a linear transformation of X and what was that linear transformation which matrix was used for the linear transformation V capital V. What is capital V? It is the eigenvectors of.

Student: X transpose.

X transpose X. So, what is happened in effect is that if I was trying to train my auto encoder with this objective function, the weights in my initial layer W would actually converge to v which are the eigenvectors of X transpose X; that means, the transformation that I have learnt this transformation which I have learnt is the same as a transformation that I have had learned using PCA. Because PCA would also have given me V into X where V was the eigenvectors of the co variance matrix and we just arrived at the same solution everyone gets it. Now we are done with the proof.

(Refer Slide Time: 16:12)



So, what we have proved is, under these specific conditions that the encoder of a linear auto encoder is linear auto encoder is equal to PCA if we use a linear decoder. If we use a squared error loss function and if we normalize the inputs to this and you understand why each of these steps was important why was the last step important.

Student: (Refer Time: 16:32).

Only then, we would have got the co variance matrix why was a step before that important because, only if it was the squared error loss we would have got that Frobenius norm objective function right? And why was the linear decoder important again the same thing? Because X minus H W we wanted it to be linear right is it fine.

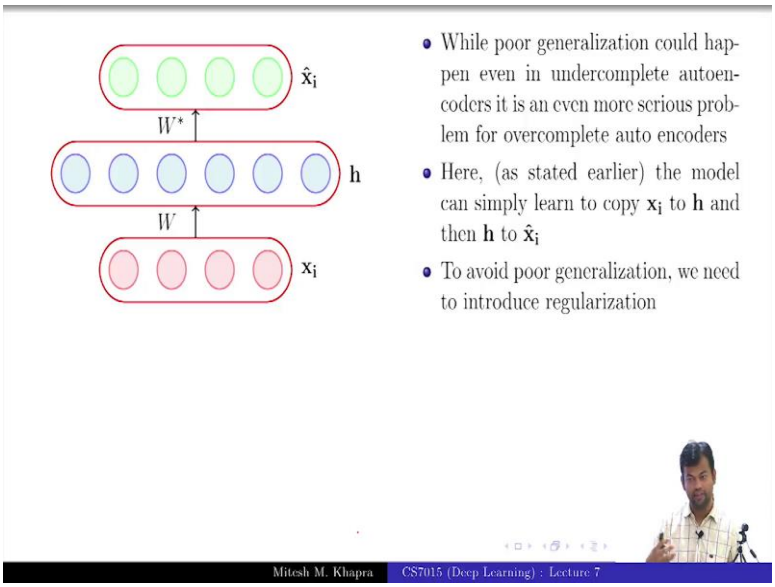
So, you see why all these assumptions were important and under these conditions, we have proved that auto encoders E equivalent to PCA.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module - 7.3
Lecture - 07
Regularization in autoencoders (Motivation)

Then we will go to the next module, where we will talk about Regularization in auto encoders, and we will talk about a Motivation for doing that.

(Refer Slide Time: 00:22)



So, poor generalization so, why do we need a regularization? People have done the machine learning course or any equivalent course, why do we need regularization? To avoid.

Student: Or enable.

Or enable generalization, right. Now in the case of an over complete auto encoder what is likely? Overfitting is likely, why is it so? What does what do you mean when you see generalization actually, can you talk in terms of training time test time and so on?

So, generalization is essentially that your are training. So, remember that at training time you are trying to solve an optimization problem, where you are looking only at the training data. So, it is quite likely that you will drive the error to 0 for the training data;

that means, you have learnt perfectly everything for the training data, right? But now it is also possible that when I give you a new test instance which you had not seen during training; that means, you had not seen instance while doing the optimization; that means, this instance did not contribute to your loss function.

Then it is very lightly that when I gave this instance, then you would get a non 0 loss or a loss much higher then what you get for your training data. Does that make sense? That is what over fitting is and it leads to less generalization. Your model should have generalize to unseen data, but it cannot do this one typical situation, where over or where generalization happens is, if you have a dash number of parameters. Now what did I ask actually?

Student: Generalization.

No, ok, if a case where a over fitting would happen is when you have a dash number of parameters.

Student: Large number of.

Large number of parameters, right; now do you see why I am saying this? What is there on the slide? An over complete auto encoder, what would it have?

Student: A large number.

A large number of parameters. So, what could it do?

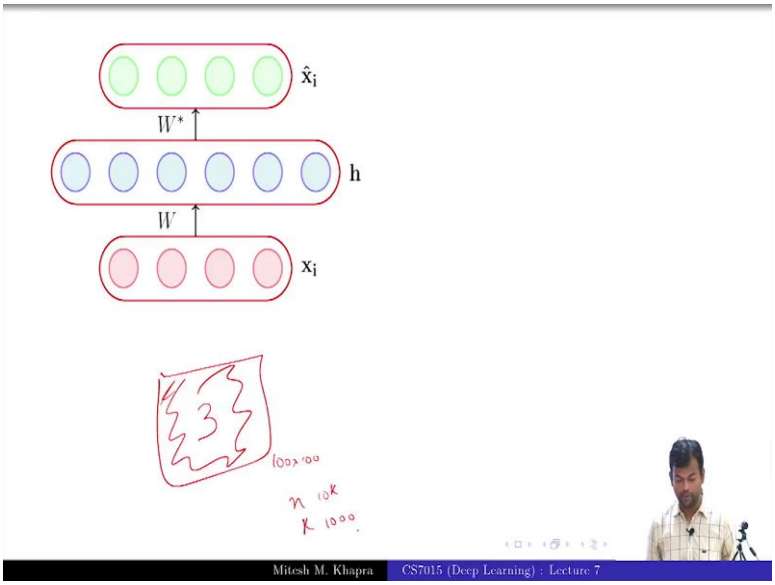
Student: Overfitting.

Over fitting, what do we do to avoid over fitting?

Student: Regularization.

Regularization, so that is why we need regularization. I have still no told you why do we need an over complete auto encoder, ok? Still that is an random variable I still need to decide. But can this happen in an under complete auto encoder also it can right? Because under complete auto encoder just says that your k is less than n, it does not say how much less it is right? So, it is it is still have and depending on a data that you are trying to model, it could still have a large number of parameters.

(Refer Slide Time: 02:58)



So, for example, let us take an example for the under complete case, suppose you are doing image classification where you have a digit 3 at the center of the image, ok. And a lot of these are white spaces. So, what is the dimension? And suppose this is a 100 cross 100 image. What is the dimension of this image input? How many if you cannot multiply 100 into 100?

Student: 10.

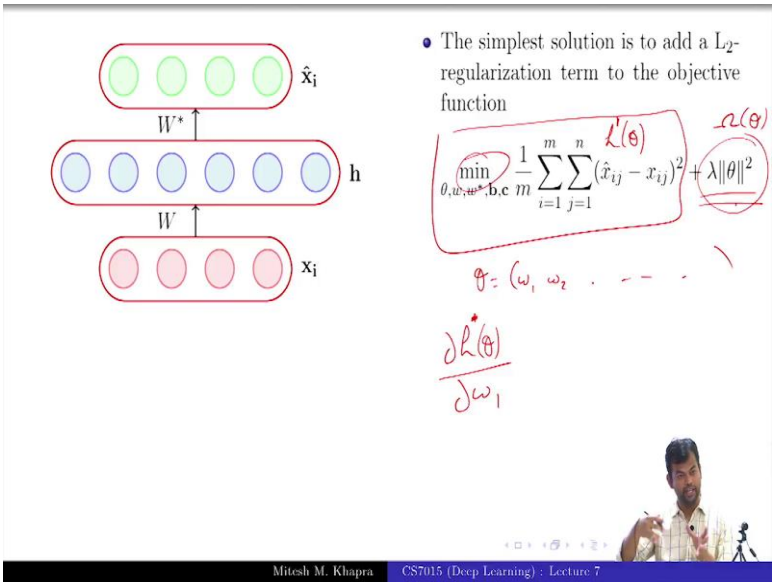
10 k right of this a lot of data is not important. So, my n is 10 k and at least by this thing that I have drawn it looks like probably only 20 percent of that is what actually captures the digit, but now if I choose k to be equal to 1000. It might still be large for this application. So, I am using an under compete auto encoder, but it could still be a situation that my under complete is still having a large number of parameters, all of get this intuition?

It is a very weird example, but still really do you get the intuition you could have a very high dimensional input, and you might think you are shrinking it a lot. But there is so much redundancy in your input that even that shrinking still leads to a large number of parameters and you could still over fit. Therefore, even for an under complete auto encoder, you could still need over a regularization.

So, fine so, that was the motivation, since the over complete case of course, the model can simply learn to copy, we have seen that, and that is why we need to introduce generalization, fine. Now what is the simplest, sorry, we need to introduce a regularization, what is the simplest regularization technique that you know? That is not the simplest L 2 regularization.

And you see why I say that is the simplest, we can take the derivative for those of you do not get it do not worry we will get to it. Or if you do not get to it do not worry.

(Refer Slide Time: 04:41)



So,ok the simplest solution is to add the L 2 regularization term to the objective function. So, this was my objective function, I wanted to minimize the squared error loss. I have added a term to this. What does this term do? What does it doing? First of all, tell me what is this quantity, theta is a?

Student: All.

All the parameters that you have; right and I am assuming that they have just put it into large vector I am taking the L2 norm of that vector. So, even you though you have those matrices, and just flattening them all out and putting them into a large vector called theta, right? So, what is happening here? I am not allowing my weights to shrink or grow, grow, because if my weights are very large what would happen?

Student: Grow, grow.

This quantity would grow. So, then I cannot really minimize this minimize this as effectively as I want, right? Why this makes sense? How many of you why this makes sense? So, I am now why am I not preventing the weights to go to 0? Ok, so, we will see this in more detail in the next lecture. This is again a basic lecture on bias variance and regularization and so on. So, we will try to arrive at a more reasonable answer for this. For now, just see that I am putting some constraints and the weights.

So, effectively and I am doing gradient descent, I am not allowing the weights to take very large values, I am trying to restrict them to a certain area. So, I am not allowing to it to explore the entire w comma b plane, but trying to restrict it to smaller values of w comma v how many of you get this intuitive explanation.

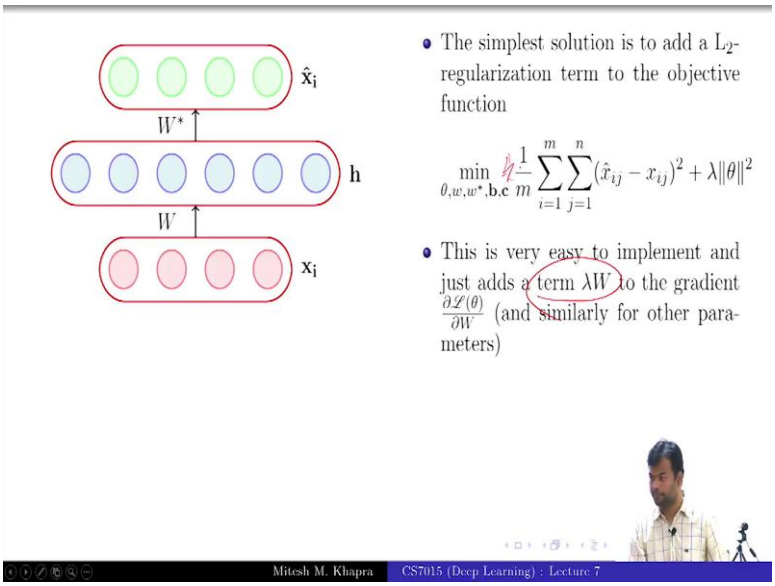
So, in other words what I am trying to do is, that I am not giving it in a freedom so that it can completely drive the error on the training data to 0. And my hope is that if I do not do this, if I do not allow it to completely memorize a training data, then it should generalize well on the test data. Is that intuitive fine ok? Now I have changed the loss function again. I have the square, I have told you how to do it for squared error loss, for the cross entropy loss and so on, but now I have changed the a loss function again. So, again I need to teach you back propagation, no what will change now? Again I need to derive with respect to the last layer.

What is the minimalistic change that is going to happen now? Just tell me, this theta is actually w 1 w 2 and so on, right? Just assume all the parameters, just flattened out into a vector, fine? And now tell me what is dou l theta by dou w 1 going to be or let us simplify things. Let us call this l theta and let us call this omega theta. Let us call this l dash theta, and then your l theta is the combination of these 2 terms. So, this derivative is going to be a sum of 2 derivatives, out of that one you already know, what is the second?

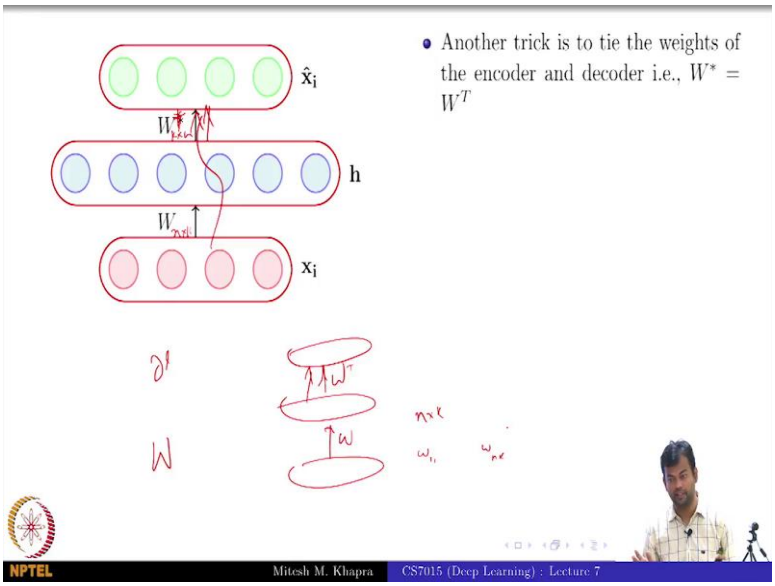
Student: 2 times lambda.

Two times lambda w 1. So, it is a very simply change to your gradient descent update rule. How many of you see that? Whatever update you will had just add minus 2 lambda w 1 to that, ok, should have been 2 lambda w, but of course, you do a half here, so it is fine, is it?.

(Refer Slide Time: 07:44)



(Refer Slide Time: 07:53)



Another trick which is typically used at least in the context of auto encoders is to tie the weights of the encoder and the decoder. How does that help? What does tying the weights mean? Now I appreciate what you are trying to say. So, one we have doing this is just say W star is equal to W transpose. You will enforce that, you actually have only one matrix W, and here you are using W transpose. Mathematically does that make sense? All your operations go through, because this is going to be n cross k, and this is going to be k cross n.

So, whatever effectively done, I have reduce the number of parameters in my network, right? I am enforcing, I am forcing this upon the network that I am not going to give you 2 sets of weights, you just learn the w's in a way such that when you use W transpose you should be able to reconstruct this. How many of you get this? Not many, ok, please ask me doubts if you do not, there is nothing very.

Student: Why is it W transpose?

Why is it W transpose? Because otherwise.

Student: (Refer Time: 08:52) Claim that.

How can you claim that that would work? Because you have no linearity's in between right no W inverse would not work, what is the simplest thing to do? Why would you want to compute an inverse? That is an interesting question how would you implement this? How would you, if there are multiple paths from a weight to the output, how do you compute the gradient? Sum it across all those paths, what is happening here? How many paths to there exist from the weight to the output? One is this direct path and then the other is another this path also. So, you just sum it across these 2 paths. Do you get that? How many of you do not get that? How many of you do not get that ok?

So, if this was W star you did not have a problem? You could just have computed dou l by dou W star and dou l by dou W. Now think of it as this, right that you have this, this is one path W, W to the output, ok. And now the gradient is just going to be sum across these 2 parts, one path is the single path and the other path is the double path. So, it is just going to be a sum across these 2 paths, oh, no, no. So, you just have one matrix W which are going to update. You do not have 2 matrices, you just have one matrix W, at 1 place you are using W, the other place you are using W transpose. But just look at it element wise right, do not try to look at it in the terms of matrices.

So, you have n cross k elements here w 1 1 to W n k, right you have to computing the partial derivative with respect to each of these, and every time they are considering all possible paths to the output, and that value is getting updated, right? And at 1 place you are using a particular arrangement of these W's at the other place you are using a different arrangement of those W's, that so, it will just remain the same, right? Is that ok?

Student: No.

No, this is for regularization, right? So, we are reducing the number of parameters by half.

Student: (Refer Time: 10:51).

Yes.

Student: (Refer Time: 10:52).

No, that I mean that also has, but that is not be the objective, we are trying to do regularization. How many of you have lost at this point? Please ask me if you have questions, really I do not mind answering. But if you just give me blank spaces, I cannot read them. So, this is used at quite a few places where you tie some weights right so that; so, effectively you are saying that learn it in such a way that it works at both the places. And you are reducing the number of parameters. So, weight tying is something which is very commonly used for regularization in the context of neural networks.

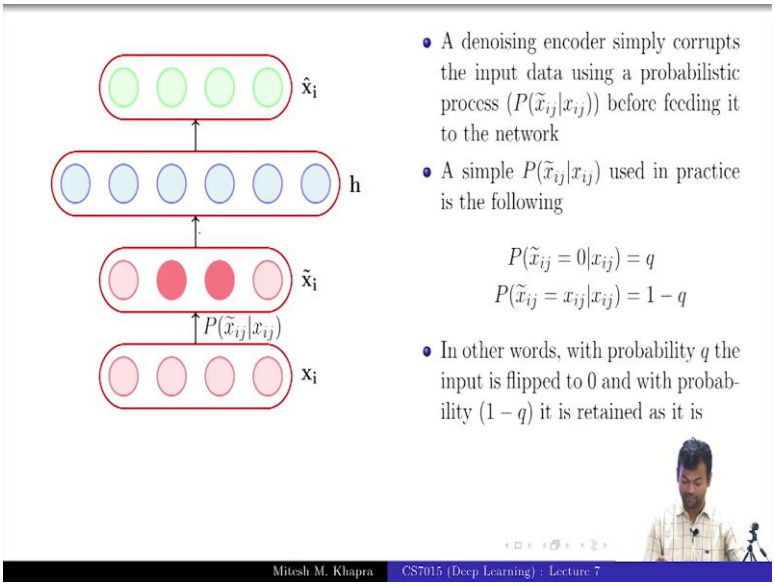
So, that is where we will end the motivation part, and it is too very simple ways of doing regularization. One is the standard known trick which is to use L2 regularization, and the other one was something special that we saw which was tying the weights you all have a lot of doubts about tying the weights.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module - 7.4
Lecture – 07
Denoising Autoencoders

In this module we will learn about Denoising Autoencoders.

(Refer Slide Time: 00:17)



So, the idea behind the denoising autoencoder is very simple, what you do is you have your original x i. Now for the minute for a minute just consider the discussion when your x i's are binary inputs ok. So, each of these red guys can be between can be 0 or 1. Now what I do is, before feeding it this input to my autoencoder; the box is the autoencoder what I do is I do a corruption.

So, the corruption is as follows with probability q I will set x ij; that means, one of these guys to 0, right? And with probability 1 minus q, I will keep it as it is ok. So, with some probability q, I am actually corrupting the data otherwise I am retaining the data as it is. And then feeding that data to the autoencoder, why would this work? Binary input case as I said just assume that the inputs are binary.

We will also see the other case, why would this work? What was our problem earlier? that was completely able to reconstruct the training data right, but at test time I had issues. Now, what I have done to the training data? Corrupted it just think for a minute what will happen; now I want someone to ask me a question, in return oh that is the corruption that I am choosing or you could flip it is what you are saying. Yeah if it is 0 change it to 1 so that is also fine. That is the question I was expecting, what is the loss function now; what is the loss function? X hat my i minus x tilde i or x hat i minus x i which choice makes sense.

Student: First tilde.

First let us the case take the case when I do x tilde i what happens in that case from this networks perspective it is still learning to memorize the training data right it just this is what it thinks as the training data, and just trying to learn that transformation right? So, it is not really helping my case, do you understand that? I just corrupted the training data that is fine, but from the networks point of view; it still gets away by memorizing this data, and that is not what I want. So, what should I do? Can anyone tell me the; I mean can everyone tell me the answers?

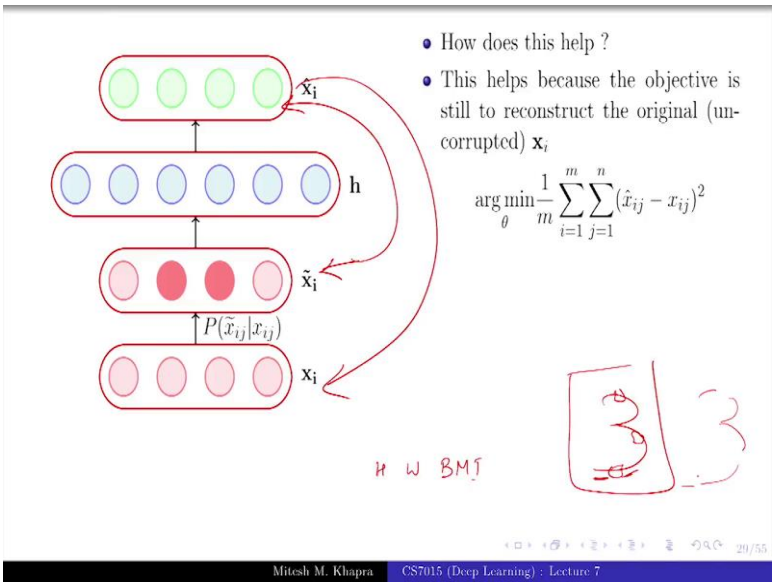
Student: Minimize.

Minimize the error between.

Student: An x i.

An x i, how many if you understand; why that should help? All of you gave the answer, but only few of your raised your hands why so, hard to deal with this inconsistency.

(Refer Slide Time: 03:04)



Besides because I am still going to minimize my original objective function ok; now can the network get away by copying the input to the output? So, input remember the input to the network is this and what I am trying to minimize this, if I just copy x tilde i to the output, will my objective function be minimized; no right? So, it does not have incentive to copy now. So, what will it have to rely on? Say a reasonable probably 20 percent is the standard right? So, even if I reconstruct I will not get 0 error I will at least get some 20 percentage.

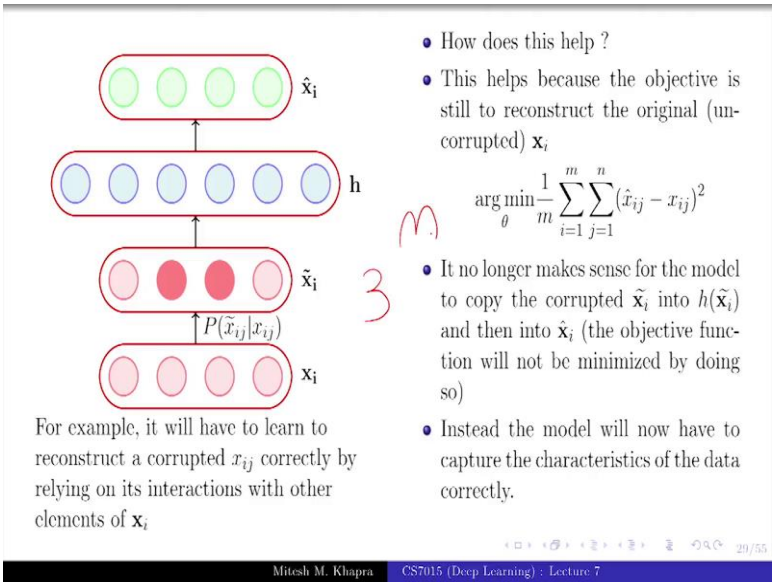
So, let us let me give you an example and then let me know if you can figure out what happens. This example will contradict something else that we have done before, but just play along. Suppose my input features were height weight and BMI and we all know that BMI depends on height and weight I hope all of us know.

Now, can you think what is happening? I am corrupting one of these inputs. And I still want everything to be reconstructed back. So, what will the network now have to rely on? It will have a now rely on this relations between these inputs also. So, again if I take my example of digit 3, I have corrupted some of these pixels right, but I still want to be able to reconstruct 3. So, it will have to be smart enough to learn that if I have seen this and I have seen this then it has to be something in between which gives me a 3, do you get the intuition.

So now I am making it is job harder. So, that it is robust to changes at test time; that means, a test time if my digit looked something like this, it should still be able to predict it as a 3 or it will still be able to learn the same representation as 3 do you get the intuition, right? So, that is what I am trying to do I am trying to somehow bring in the corruptions that I would expect a test case and trying to make the model more robust.

It can no longer get away by memorizing the training data because I am not feeding it the correct training data, it has to do something smarter than that, everyone gets this. I will come back to your question everyone gets this please raise your hands yes, yes this is all under regularization no this is regularization no so at that case I have already made that overfitting can happen in an over complete as well as under complete autoencoder, everyone gets that right? I show that example where it could happen in both the cases. So, my figure maybe over compete, but it can just happen in any of these is that fine.

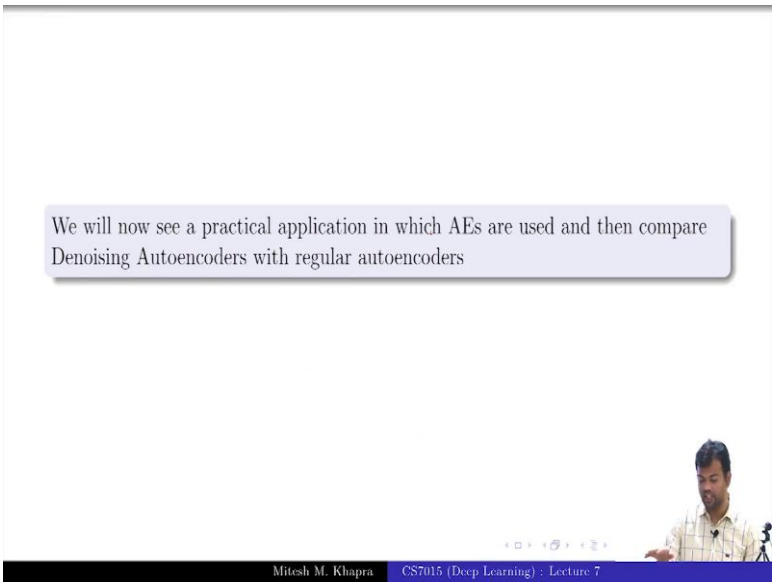
(Refer Slide Time: 05:44)



It no longer makes sense for the network to just start copying the input data.

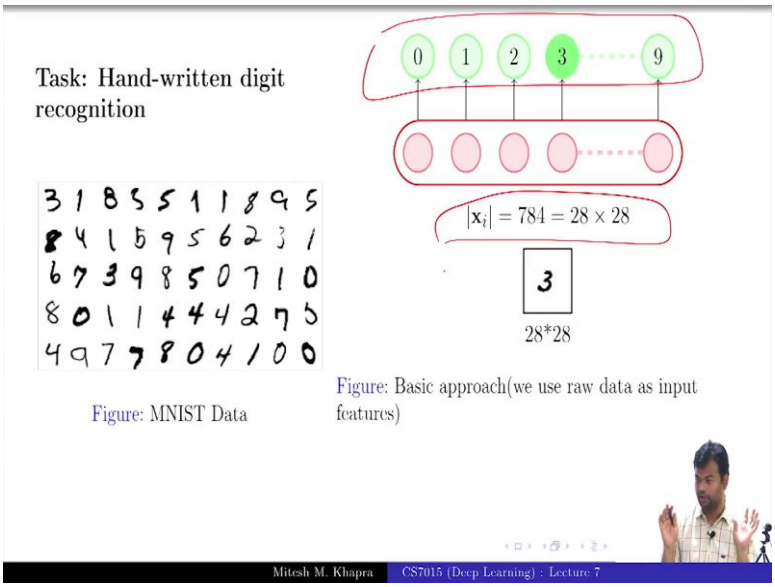
Different kinds of noises means yeah. So, let me try to answer that right? So, what probably you are trying to say is that all my input images were 3 vertically written. I added some noise and managed it, but now at test time suddenly you show me a 3 of this kind, like that will not work also. That is what were your question was a different types mean different values of the noise 20 percent 25 percent and so on.

(Refer Slide Time: 06:16)



So, we will first see practical application in which autoencoders are used and then compare it to denoising at autoencoders. So, this the next few slides for those of you may care is also a small answer to the difference between machine learning and dp.

(Refer Slide Time: 06:36)



So, suppose you are given this task, which is handwritten digit recognition I see everyone paying attention. Now I should say this before every slide ok? So, this is the task handwritten digit recognition you are given some data, where you want to classify the digits into one of these 10 glasses. The traditional machine learning approach to this

is we just construct a feature vector this is a 28 cross 28 image. So, I guess 28 cross 28 pixels which is 784. I treat this as a feature vector and feed it to any of my machine learning algorithms say SVM or multi class SVM or logistic regression or any of these right and do a classification based on them. This is what you would have done in your machine learning course if I had given you this assignment right?

(Refer Slide Time: 07:25)

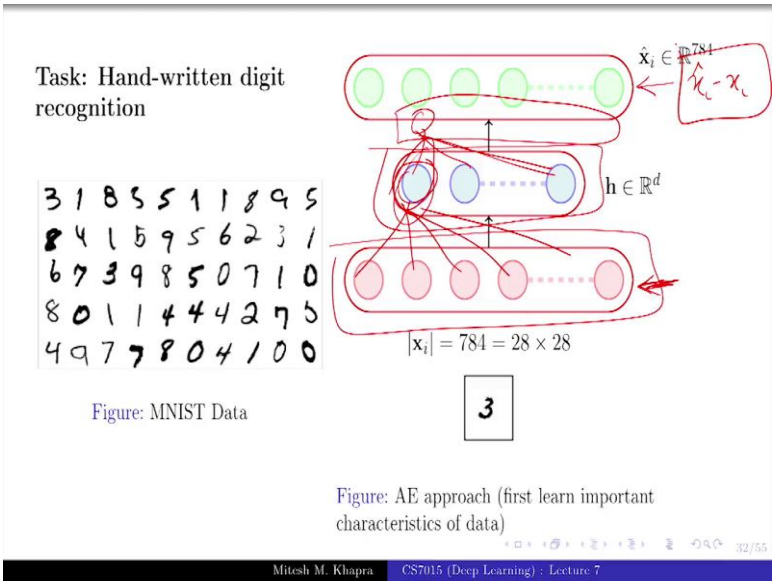


Figure: MNIST Data

Figure: AE approach (first learn important characteristics of data)

Now, the autoencoder approach or in general the deep learning approach would be you take this data which is the original feature representation that you had. There is no engineering feature engineering happening here right, ideally I want to have features of the form that if pixel 25 comma 30 was black and if pixel 30 comma 20 was also black then probably I am drawing a curve somewhere. So, it could be one of these curvy digits and not one or any of these 7 or any of these things right? So, you want to do some feature engineering.

So, typically in machine learning what you do is; you start with these 784 features you observe a few things and you have these handcrafted features added on top of these right? So, you will add some more features to the data. Now the deep learning approach is that you let you also learn the features on their own. So, how did we learn these features? We took this original input we passed it through the an autoencoder which captured some of these relevant characteristics.

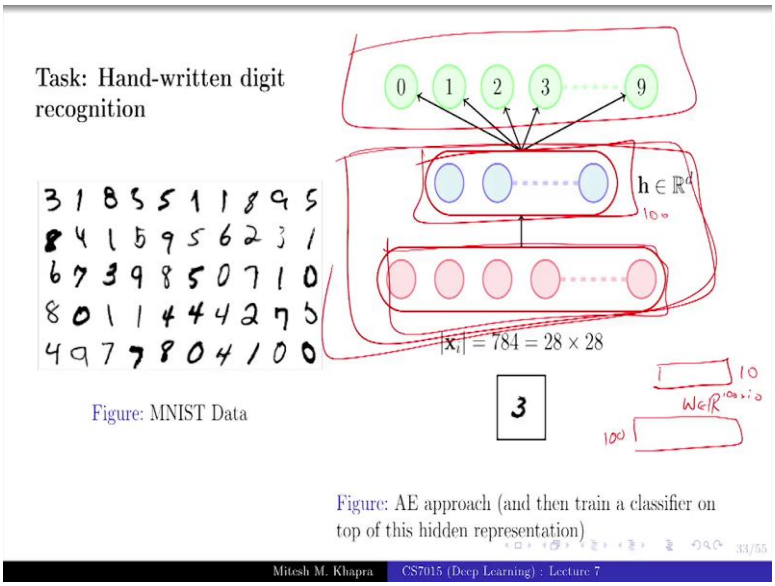
The differences we do not really know what these relevant characteristics are; that means, you and I cannot read them and make sense of them. I cannot say that this pixel is actually capturing the interaction, oh sorry this neuron is actually capturing the interaction between my 700 pixel and 710 pixel, I cannot do that. I could have handcrafted those features. If I believe that all my data is around the center, I could have handcrafted some features which say that capture the interactions between those that is what you do in machine learning.

Here you are trying to learn the features also on their own right, what would happen if I add one more layer to this autoencoder? I would learn even more complex interactions between these features. So, this neuron is actually learning interactions between all the input neurons.ok I add one more layer here again this neuron will learn all the interactions between these abstract representations, right? So, I could learn more and more abstract representations of the input. So, I am not doing feature engineering I am just throwing data at the network and I am assuming that it will learn better and better representations.

Now, I am doing this in the autoencoder setup where actually I am trying to optimize the objective function of minimizing this loss and of course, the squared of this loss is just fine. So, first what I will do is; I am not happy with my original 784 dimensions. So, I train a autoencoder to learn some k dimensions which are good. I know these are good because they are able to reconstruct the data perfectly to a certain extent right of course, because you add regularization it may not be perfect, but it captures the essence of the data, you get that.

So, I have better dash representations now feature representations right? My original feature representation was 784 I have come up with some better representations. Now what will I do; was my task to learn feature representations, what was it? Classification right, so what will I do now is I will I have learned this much from the autoencoder.

(Refer Slide Time: 10:18)



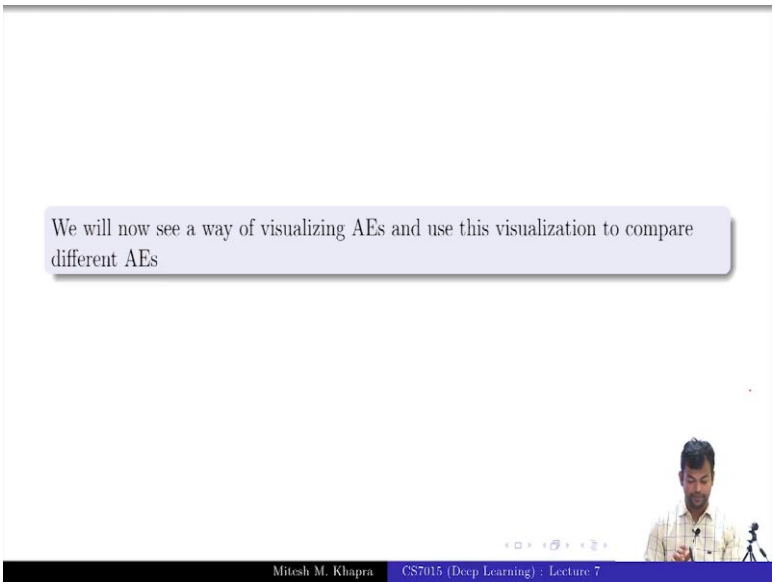
I will throw away the last layer I do not care about the last layer, what I care at the last layer is a classification problem, right? So, I will construct a new neural network, where the first 2 layers of the network are the same as what I learned from the autoencoder and on top of that I will add an output layer, and now I will try to train this network, how many of you get what is happening here? Those of you do not get it can you ask me some questions let me just try to answer on my own it is like playing chess with yourself.

So, this is my original input 784 dimensions, what I have learned with autoencoders is a smarter representation of this data ok. Now one simple solution that I have is I have this 100 dimensional data suppose this is the representation. So, for all the training examples instead of using that 784-dimension data and feeding it to a multi class SVM, what I can do is; I can first compute this 100 dimensional representation and feed that to a multi class SVM, is that fine and you see that should work better in practice because I have reduced the dimensions, I have reduced the dimensions smartly.

And now I can train this network is this fine. All I am saying is instead of a multi class SVM. I could also have a neural network right; I could feed that representation to a neural network. So, what would that neural network look like; 100 what are the parameters here? W belonging to 100 cross 10 how many if get it now ok. So, this is what I could have done.

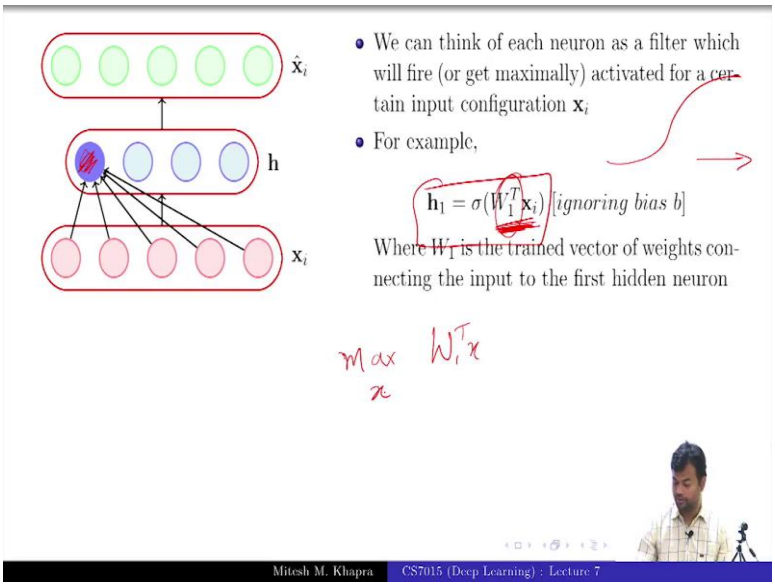
So, I have learned a better feature representation and now I am using that representation to learn my classifier. If I do this in an end to end manner; that means, my feature representation is also came out of a neural network. And my classifier is also a neural network then I have a complete end to end solution for this you get this.

(Refer Slide Time: 12:14)



Now, we will see a way of visualizing this and then we will make some observations from the visualizations. So, first let me tell you what the visualizations is.

(Refer Slide Time: 12:23)



So, I am returning to the autoencoder setup. So, I had this input and I had this h dimensional or k dimensional hidden layer. Now I can think of each of these neurons as something which gets activated for a particular type of input, is that fine; what do I mean by activated it is output would be?

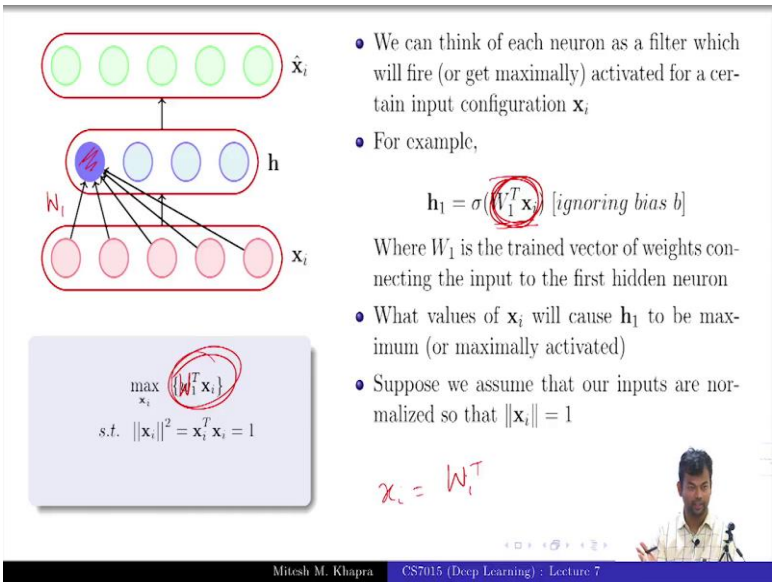
Student: 1.

Remember this is a logistic neurons that we are talking about or even Tanh neurons the output would be 1. So, it is the maximum output that you could gain fine. Now so for example, h 1 is equal to sigmoid of this when would this fire when where W 1 transpose x i is very high right when you are in that regime where the sigmoid flattens right, this regime ok; when it is very high.

So, I want to be able to maximize W 1 transpose x i, do you get this? I want to be able to maximize this I want to find my W 1 transpose is fixed now because I have trained the autoencoder. I have got these weights this is all post mortem, right? I have trained the autoencoder, I have got these weights. Now I want to find an input which will cause this particular neuron to fire.

So, what is my max what is my optimization problem maximize just help me out, maximize W 1 transpose x let me just call it x, and the optimization is with respect to x right because I want to find the x which maximizes this quantity my training is done, I do not no longer care about changing Ws my training has been done I am interested in finding x's which will maximally fire this.

(Refer Slide Time: 14:08)



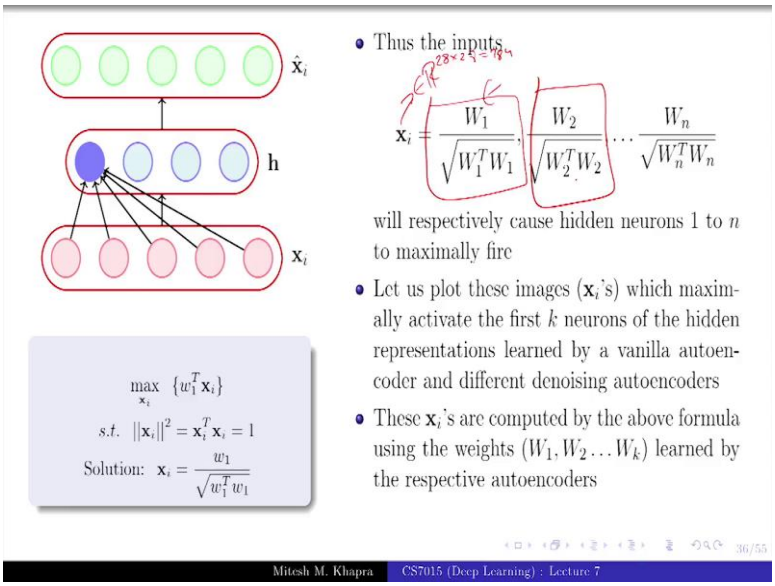
So, and I am going to assume that all my inputs are normalized this just makes some analysis easier and remember that normalization is always ok, you always do that. So, this is the optimization problem that I am interested in solving. What is the solution to this; how many if you can solve this? No I want to find the x i.

Student: (Refer Time: 14:29).

Now, I have trained the autoencoder, now I have known all these the one I am considering one column of the matrix W 1. I want to see what is the input that I should give. So, that I am sure that this neuron will get activated and I know that this neuron will get activated if I maximize this quantity right?

So, I want to maximize that quantity and find an x such that it will get maximized. I was just hoping that no one brings in eigenvectors W 1 is a column it is not a matrix just try to work it out. What is this? This is a dash between W and transpose and x i dot product, when would the dot product be maximized? When they are both in the same direction right; that means, you know the direction is going to be x i is equal to and what did I want the norm to be; now do you get it fine.

(Refer Slide Time: 15:26)

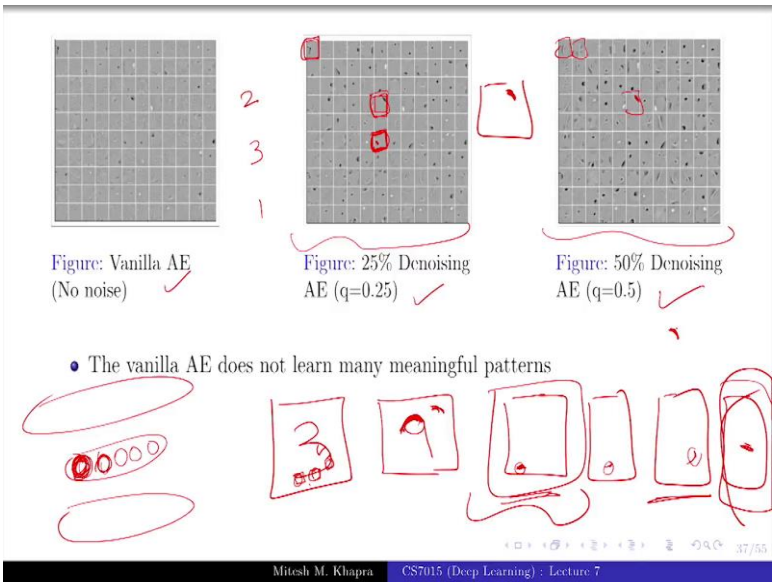


So, the solution is going to be this is fine W 1 by the norm of W 1. So, just remember that this quantity is going to get maximized when the dot product is maximized the dot product is maximized when both x i and W 1 transpose are in the same direction right so; that means, x i should be in the same direction as W 1 and I also wanted this constraint that x i should be the norm of x i should be 1. So, I am just dividing W 1 by the norm of W 1.

So, I know now what is the input I should feed to the network so, that one of these neurons fires. Now what I am going to do is I am going to plot the xi's which maximize each of these neurons I am going to consider some 100 neurons in the hidden layer. And I am trying to find out the input image which is going to maximize or which is going to cause each of these neurons to fire, do you get what I am trying to do? Even though you do not get why I am doing it, but do you get what I am trying to do, ok?

So, what am I going to do is; this is a vector, right? So, I am just going to try to plot this as an image of the appropriate dimension.

(Refer Slide Time: 16:41)



And this is what I get with a vanilla autoencoder there was no noise this is what I get and this is for the MNIST digit data set right? So, my data is 2 3 1 and so on digits, this is what happens when I get 25 percent nice, and this is what happens when I get 50 percent, what do you understand from these figures? Remember that each of this is the figure which caused one particular neuron to fire is that clear each of these is a trigger which caused one neuron to fire.

One image yeah one box corresponds to one column yeah. So, it is just that the dimension of the column is again 28 cross 28. So, I am just plotting it as a 28 by 28 image. So, I will just let me just clarify that is I think that is what I said yet. So, what is the dimension of this? In fact, you just know this right this dimension of this is 28 cross 28.

So, I can just take that vector and again plotted as a 28 cross 28 image. So, what I mean is this is 784, right? So, x i is a 784 dimensional vector I am just taking it as a 28 cross 28 image and plotting it because my inputs were actually images. So, I am just plotting those images fine. So, at least you see, what I am doing here and what I am telling you is that each of these boxes that you see corresponds to one of these images. So, I had images x 1 x 2 up to x k such that; each of these caused the k'th neuron to fire, ok? Now what are you seeing here I mean what how do you make sense of what you are seeing

and remember in the MNIST ok sorry. So, let us try to forget all this neural network and everything and let us just try to see yes the weights would be.

Student: More distinct.

No why do you say the weights are more distinct yeah, but on average you would be still reducing it right ok. So, let me just explain what is happening then we can come back to this. So now, we have this set up we had some input. We had a certain number of neurons here and then we had the output ok, this is what our neural network was trying to do.

Student: (Refer Time: 18:53).

Now, let us take this task of recognizing a digit. Now how do I actually recognize a digit if I want to distinguish between a 9 and a 3, I would try to see if there is a curve in these positions and it is not there in this, hence this is a 3 this is a 9 that is something roughly like that right? So, in other words I am now I have given delegated so; that means, what I do is I think of 3, as a combination of you get the idea as a combination of these images with these strokes right? So, this is actually this stroke this is actually this stroke, this is roughly this stroke and so on you get the idea.

So, I think of 3 as a combination of many of these strokes, right? Now what I would like is if this guy could detect one of these strokes, right? The other guy could detect one of these other strokes right? Now you see that some of these strokes are shared across digits for example, all these strokes here look at the digit 9 these strokes gives common to 3 and 9 both right, but some strokes would be missing for 3 some strokes would be missing for 9 and you would have extra strokes in both of these. So now, each of these neurons could actually recognize these strokes; then a combination of the information that each of these neurons is capturing could help me decide whether it is a 3 or a 9, how many of you get that intuition?

Student: (Refer Time: 20:23).

So, I would like each of these neurons to detect certain strokes ok; that means, I would like this neuron the first neuron to fire for an input like this, where there is a stroke at the bottom. I would like some other neuron to fire for a different input whether there is

stroke here. Now can you relate this to what you are seeing in the picture, in the second and third picture; this neuron is firing for inputs which would have a stroke at the corner right and you see different neurons are firing 4 different strokes. So, each neuron is trying to capture something relevant. And together now I could combine them to get the final output, how many of you see this; how many of you do not get this?

So, to ask questions otherwise I cannot really help it, how many of you want me to go over this again? Which part yeah so let me just repeat what each of these boxes is right? So, each of these boxes is the image which causes the k'th neuron to fire right? So, remember I decide I came up with this that this is the input which causes the second neuron to fire, what was the dimension of this input? 28 cross 28.

So, I am just plotting that 28 cross 28 input right, and I am realizing that this input seems to be something which has a dark spot here right? So now, just related to the analogy that I am trying to give at the bottom that this neuron fires for inputs which have a stroke here, that is that is capturing and there are other neurons which are trying to fire for other strokes.

And I would want these neurons to capture different strokes. So, that together they captured all the information in the image and helped me decide that a combination of these strokes gives me a 9 a combination of these other strokes gives me a 3 is that clear now you also.

Student: Yes, sir.

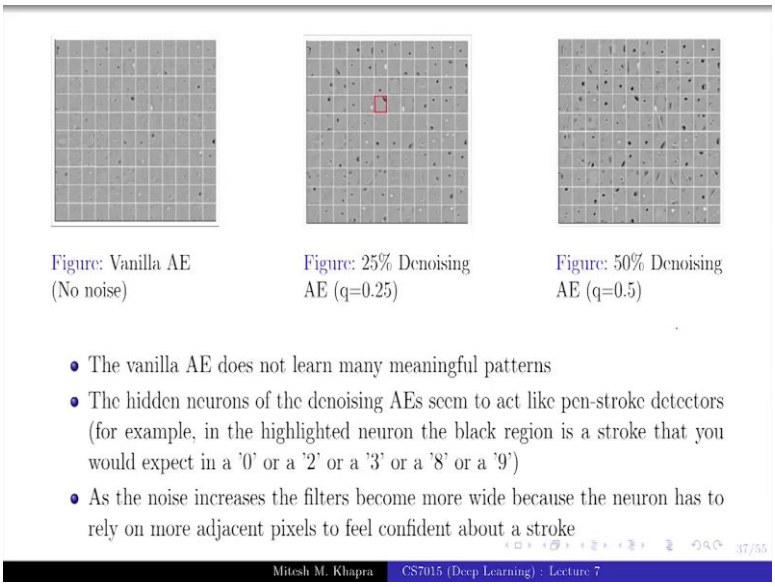
So, yeah. So now, the thing is this right the again the same thing you could learn to reconstruct the output, but you may not capture the important characteristics in the input, right? So now, as you keep making it is job harder it has to rely on capturing these important characteristics in the input, right? And actually if you look at the difference between the second figure and the third figure right let us look at the same guy here.

So, you see that this is actually thicker and wider, the stroke that you see here is thicker and wider. So now, it is actually relying on more neighborhood information to fire it is not firing just for this stroke, but it is fighting for a larger stroke it is also requires more neighborhood information because you are corrupting the pitch.

So, it has to rely on information from the other guys, the same example that I gave for height weight and body mass index right the same thing holds here. I have corrupted a lot of inputs. So now, it will fire only if it gets a lot of information from the neighboring inputs also is that fine ok? And I now coming back to your question, yeah I do realize now what you are saying that the weights are actually becoming larger yeah, it makes it more robust, but again.

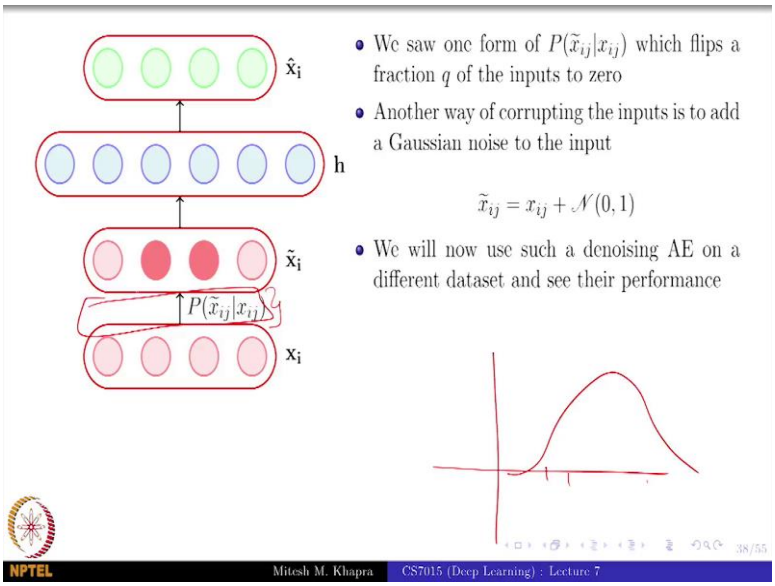
So, regularization just does not always mean that your weights have to be small right that is one way of constraining or regularizing, but this is another way of regularizing where you are making it more robust, but it does not necessarily need to lead to the same solution where your smaller weights does that make sense, it is ok for most of you any please raise your hands if this?

(Refer Slide Time: 23:59)



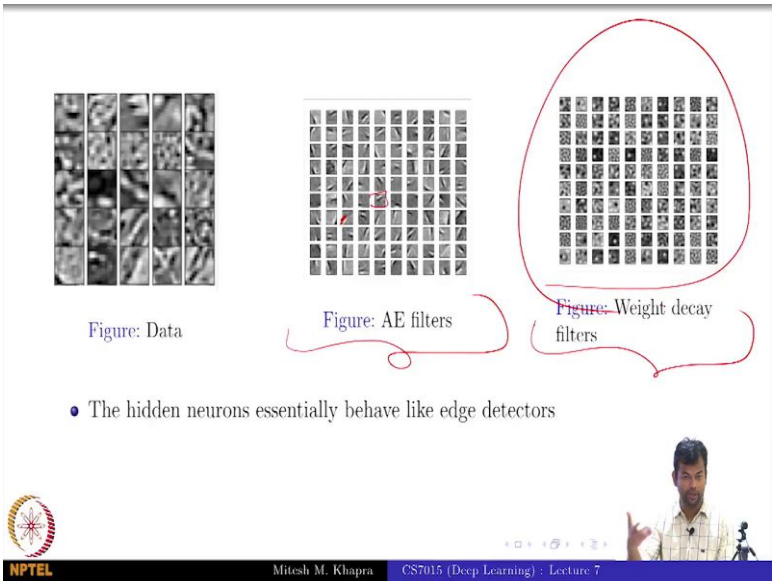
And this is same thing that I have written here.

(Refer Slide Time: 24:02)



Now, we saw one form of this function ok, which was just flip the input if the output is just corrupt the input right, you could also add a Gaussian noise. So, you could take the input add a Gaussian noise to it with 0 mean and then again try to reconstruct the original input back is that fine. So, you could just use different noise functions to do this. So, we will now see such a denoising autoencoder where we have actually added a Gaussian noise, instead of the 0 1 noise or the corruption that we were doing.

(Refer Slide Time: 24:39)



Yeah so the purpose of this particular example that I am giving is to compare an autoencoder which is regularized by adding this Gaussian noise, with an autoencoder which is regularize by using weight decaying right the L2 regularization. So, L2 regularization is also known as weight decaying because you kind of decay the weights right you force the weights to be small.

So, what they showed is that with Denoising Autoencoder using a Gaussian noise, you actually learn something known as edge detectors right? So, you see all of these are trying to detect edge, again the same thing is happening I am plotting the images which will maximally cause a particular neuron to fire and it looks like all these neurons fire for different edge patterns in your original data.

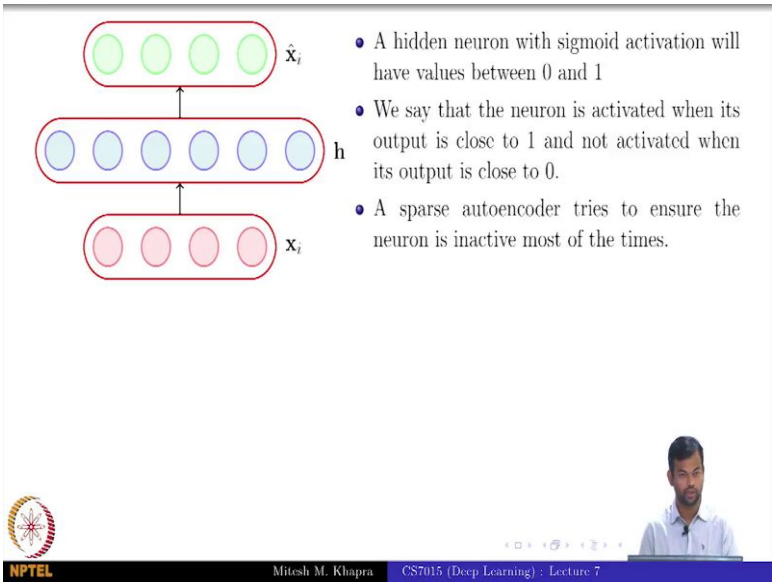
So now they are capturing all the edges in the data, and the combination of these edges should tell you what your final class is, ok? And this seems to work much better than the weight decay filter which is not really capturing any regular pattern ok. So, this is just an empirical evidence that an autoencoder with a Gaussian noise seems to do better than autoencoder with the L2 regularization.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 7.5
Lecture – 07
Sparse Autoencoders

So, in this module we will talk about Sparse Autoencoders.

(Refer Slide Time: 00:17)



Just some concepts before we jump into the actual way of doing this. So, hidden neuron with sigmoid activation will have values between 0 to 1 and you say that the neuron is activated when this output is close to 1 and it is not activated when its output is close to 0 ok. Now, a spare encoder tries to ensure the neuron is inactive most of the times, what is that mean?

Student: Close.

It is close to 0 for.

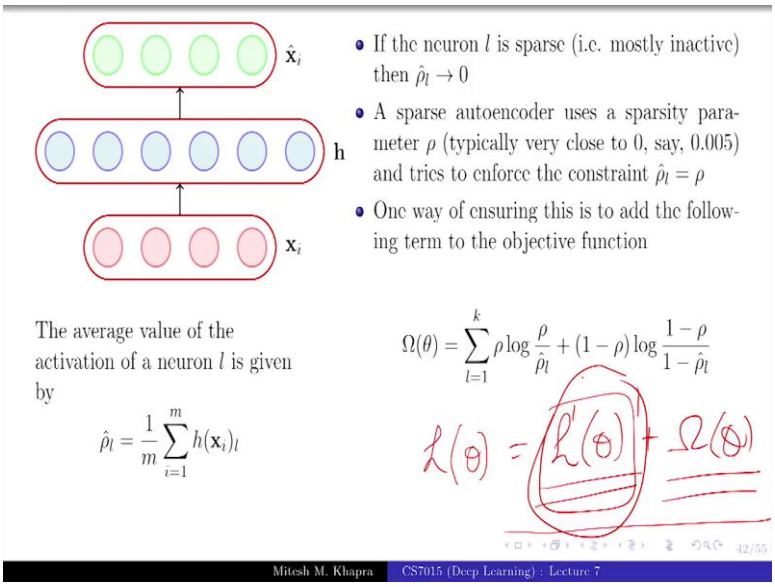
Student: Most of the.

Most of the.

Student: (Refer Time: 00:47).

Inputs right? So, I am passing a lot of inputs to it, it will try to ensure that it is close to 0 for most of the inputs. So, in other words what does it trying you ensure? I am looking for the word average. The average activation of a neuron is close to 0 does that make sense is that fine.

(Refer Slide Time: 01:06)



So, this is on, what you see on the left hand side, this is how you would compute the average activation of a given neuron, you have all the m examples you see what the activation was for each of these and take the average right?

Now, if the neuron is sparse then the average activation would be close to 0 is that fine, this is all just different ways of saying the same thing. Now, a sparse encoder uses a sparsity parameter say rho and it is very close to 0 say 0.005.

And it tries to enforce the constraint that on average the activation of any neuron in the hidden layer should be equal to rho, which is again close to 0. Now, can you think of a, this is all fine in plain English right, you understand what we are trying to do. First of all tell me why does this makes sense? What is it that you are trying to ensure? Over fitting happens because there is lot of dash.

Student: Parameters.

Parameters slightly abstract it out.

Student: Memorization.

Lot of?

Student: Memorization.

Memorization ok, lot of freedom right, I mean the weights have a lot of freedom to move where ever they want to do, whatever they want to do such that they can just drive the training error to 0. What have we done to that freedom now?

Student: We are restrict. (Refer Time: 02:22).

We are restricting them. So, any kind of regularization always tries to restrict this freedom that the parameters or the network have in general right, and there are different ways of restricting this freedom. You see that this is one of those ways right; you are trying to ensure that on average the neuron should not fire. So, it is clear that this some kind of regularization, any one has a doubt with that? No.

Now, the second question is taking slightly more on this right, it is I can just move ahead and I have convince you that this is regularization. But can you think of bit more and see what is actually being tried to achieve here, what are we trying to do? How many of you get that or at least could here that first of all, only the second row ok. So, yeah how many of you can think about this, like what is it trying to achieve?

Student: (Refer Time: 03:13).

Right. So, on average neuron is going to be inactive, that means, where ever it is active it is really going to capture some relevant information, right? So, it is going to be active whenever it is active it is going to adhere to certain patterns. So, we are ensuring that each of these neurons are just a very few patterns and it has discriminative power in that sense, do you get that?

So, now if; that means, if I show it a 3, if I show it a 2, if I show it a 1 every time if the neuron fires, when there is no discriminative power in that. But now, if I ensure that the neuron fires only a few times it will try to fire for meaning full patterns. So, it will try to fire for a curve or a curve in the between as you have it in the case of 3 right, you have this cusp in the between, in the middle. So, it will fire for some kinds of pattern.

So, that is what the hope is, it is not just like adding some math and adding some regularization, but at least there is some intuition behind that, how many of you get that intuition? Ok good. And now can tell me a way of putting this, everything English is fine, intuition is fine, but how do convert this to a mathematically equation?

You want to ensure that rho hat l is equal to rho, there will of course, be different ways of doing this, the way these guys do it by adding this term to the loss function. So, remember your loss function is always going to be L dash theta plus omega theta right? Where omega theta does the regularization and L dash theta is your regular loss, which would be the squared error loss or the cross entropy loss or whatever loss you are dealing with right?

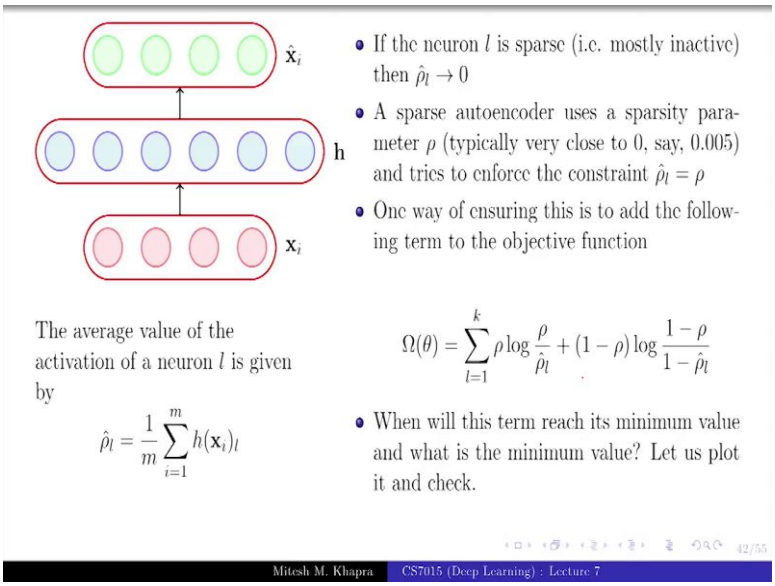
So, remember this term is always there, but the reason I do not bring it up so often is because we have already dealt with it. We know how to compute the gradients, we know how to do the back propagation and all that. And now since your final loss is just a sum of these two terms, I know how to deal with this and I know that gradients are additive so I just need to deal with the second term. That is why I am only focusing on omega theta, l theta has been dealt with, is that fine?

Now, this is what omega theta is, why does this make sense? When would this take its minimum value when rho is equal to?

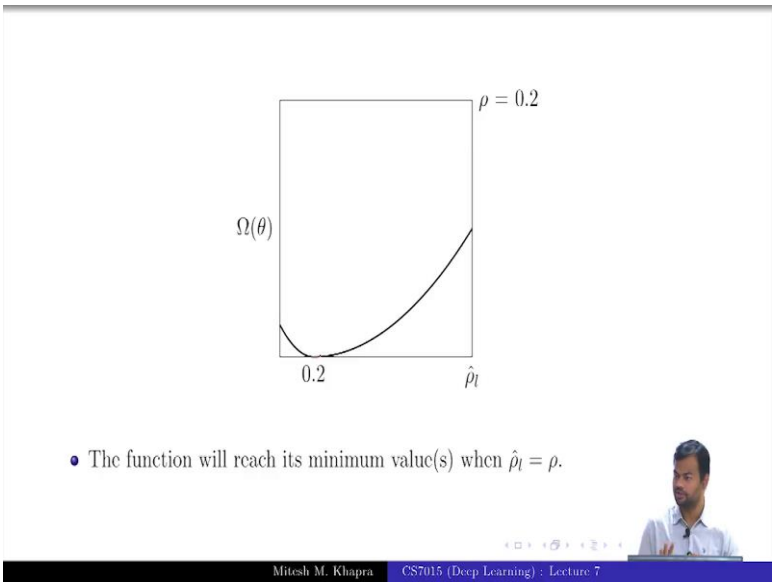
Student: Watt.

Watt, everyone sees that how many of you sees that? Please raise your hands ok, fine let us plot it and check actually, right.

(Refer Slide Time: 05:23)



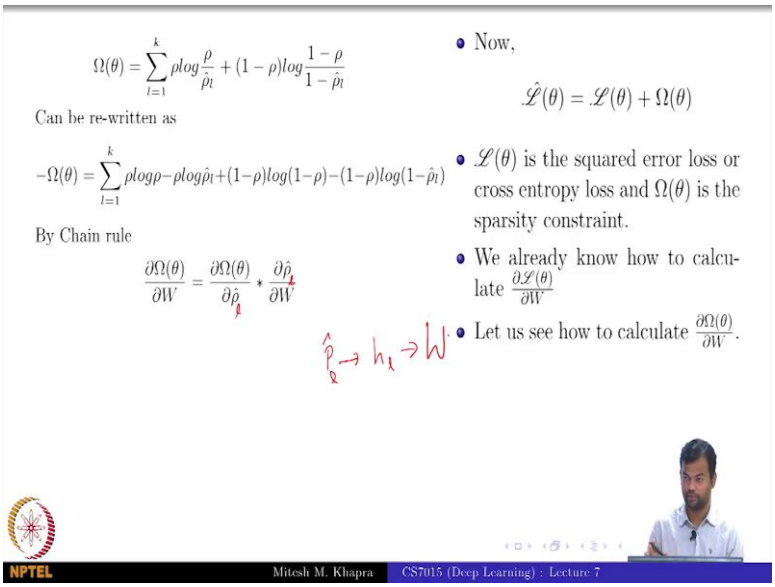
(Refer Slide Time: 05:27)



So, this is how that function looks like. So, I have plotted the function which I have written here for a of course, a single k right and my rho that I have taken is 0.2 and if I plot that function for different values of rho hat l, it will reach the value 0 only when rho hat l is equal to point. So, again go back and plot this and check and it is actually clear from the equations itself that it will be minimized only when rho hat is equal to rho l, right?

So; that means, this is a genuine, I mean this is a reasonable thing to do, we would think of other ways of doing that and I am sure you can, but this is also a reasonable way of doing this.

(Refer Slide Time: 06:08)



So, now our last function is as I said it is going to be a combination of two values, l theta is a normal squared error loss that we have been dealing with and omega theta is this sparsity constraint that you have added.

Now, you already how to calculate the first term, what are we interested in now? So, you see that this pattern will keep repeating right so now, you can do whatever you want your loss function. You have this generic frame of called the back propagation algorithm and you know that a last part of that back propagation algorithm is going to remain the same, right? Only thing you are changing is the output layer or the loss function.

Just need to compute something there and the rest of it will remain the same how many of you get this general idea? And also appreciate it right? That is why this is a very powerful frame right, you can just make minor tweaks at the top and you are rest of the code has to remain the same.

So, you can actually go back and try out these regularization terms in MNIST assignments right, if you really want to see what happens ok? So, now, this is what

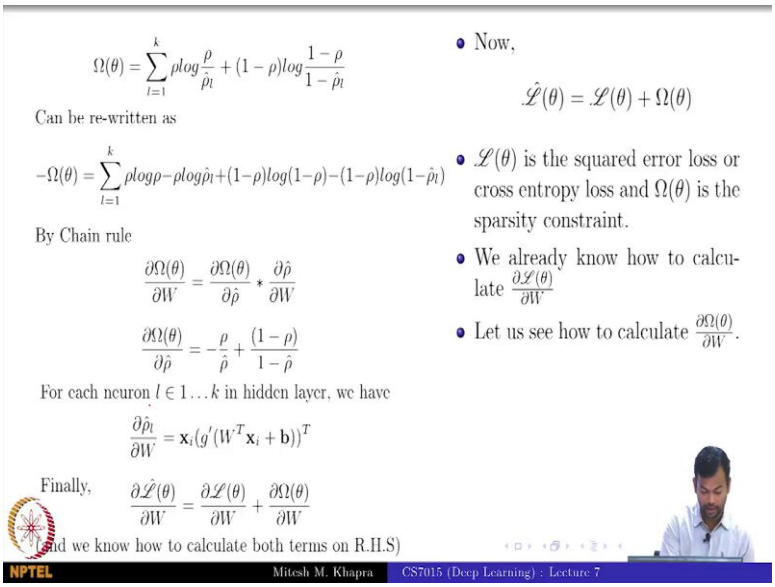
omega theta is and now what I am going to do? It can be rewritten as this, that is obvious just expanding out the law of function.

And by chain rule this is what I get, now unfortunately the rest of the slide there is an error the TA's please note this I can kind of overlooked this ah, but I will just convey the idea, right? So, you would want to do something of this sort, everyone agrees with that, remember what is rho hat? It depends on, sorry rho hat l it depends on.

Student: (Refer Time: 07:36).

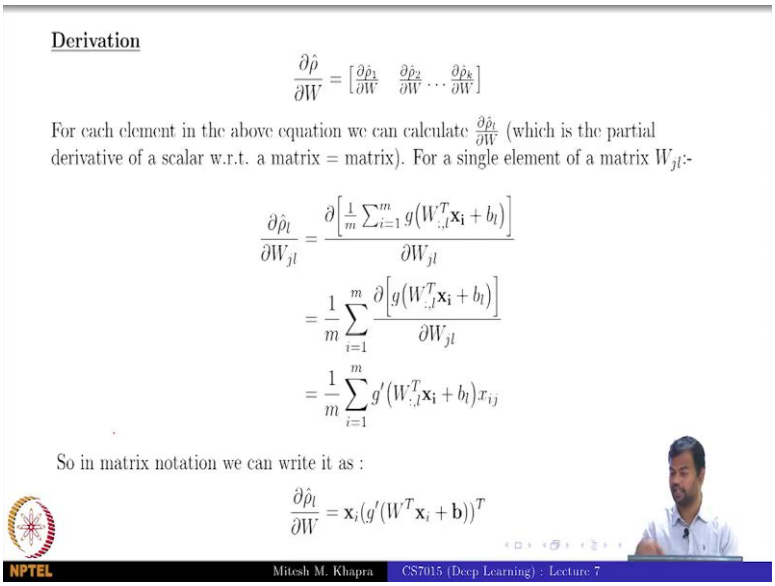
h of l right it is the average activation of the l'th neuron and this depends on some of the weights. So, that is why this chain rule makes sense and now how to compute this? There is an error on this slide but you have done enough gradients in the class for me to have confidence that you can do it on your own, everyone is confident that they can work it out on your own?

(Refer Slide Time: 07:58)



So, I will skip this, we will fix these errors there are some summation and other terms missing here and the second part is actually correct which has been derived on the next slide.

(Refer Slide Time: 08:05)



But I would not go over this, this is there are the slides again go back and look at it. How many of you are confident that you can do this on your own? Please raise your hands yeah because we have done enough of this in class right?

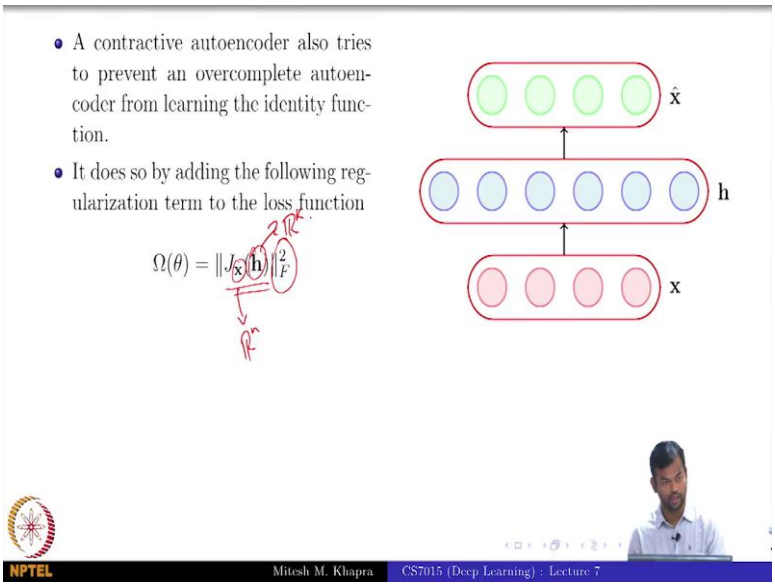
So, you can, you should be able to it no if you are not able to do it then I am not doing a good job at teaching you right? So, you should be able to do it now fine and we will fix these errors; so, TA's just remind me after the class. So, everyone gets the general idea, you find a loss, you find a constraint, you define it with omega theta, find out the derivative of that with respect to your parameters and just change your gradient descent upgrade tool accordingly.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 7.6
Lecture – 07
Contractive Autoencoders

So, with that we will move on to something known as Contractive Autoencoder. So, this is yet another type of auto encoders again with the same aim that you want to do some kind of a regularization.

(Refer Slide Time: 00:25)



So, it again tries to prevent and over complete auto encoder or even an under complete auto encoder for that point, from learning the identity function.

So, it does not allow you to simply copy the inputs to the outputs, that is what it is trying to learn. And it does so by adding the following the regularization term to last function and the way it does this is by defining the following regularization term ok. What is this term? Ok, let us see some things which we already know, what is this? Frobenius norm of some matrix, what is this matrix?

Student: Jacobean.

Jacobean, what is the Jacobean?

Student: (Refer Time: 01:00).

What are the two variables here that you see?

Student: H.

H and?

Student: X.

H is a scalar matrix vector.

Student: Vector.

Vector; x?

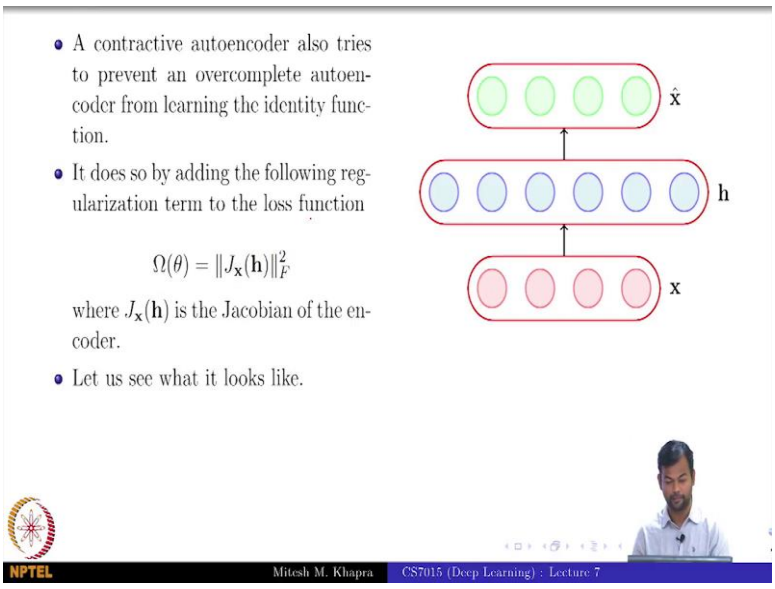
Student: Vector.

Vector right? So, it is some function between two vectors ok and it is a matrix. So, take a guess how many entries would not you have, if x is R n and h is R k.

Student: N cross k.

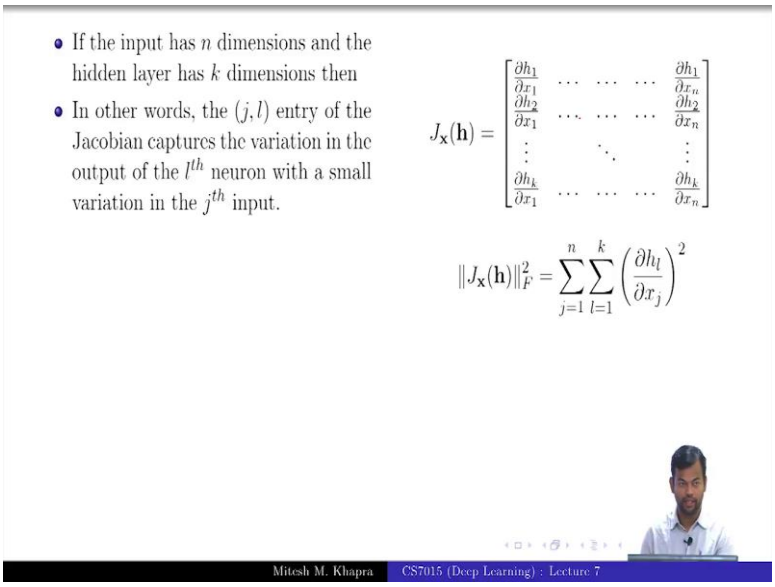
N cross k, even if you do not know what the entries are you are able to guess that it is going to be a n cross k matrix right?

(Refer Slide Time: 01:34)



Now, let us see what this n cross k matrix looks like ok.

(Refer Slide Time: 01:37)



So, it has, the input has n dimensions and the hidden layer has k dimensions. So, this is what the Jacobean looks like.

What is the first column? If the partial derivative of every neuron in the first hidden layer with respect to the first input right and now you can see what the other columns would be. This is what the Jacobean is, this basically the derivative of h with respect to the vector x, answer is just you are taking a derivative of a vector with respect to another vector you will get a matrix as the output ok. Now, what does the jl'th entry here capture actually?

Student: (Refer Time: 02:12).

What does a derivative capture?

Student: (Refer Time: 02:14).

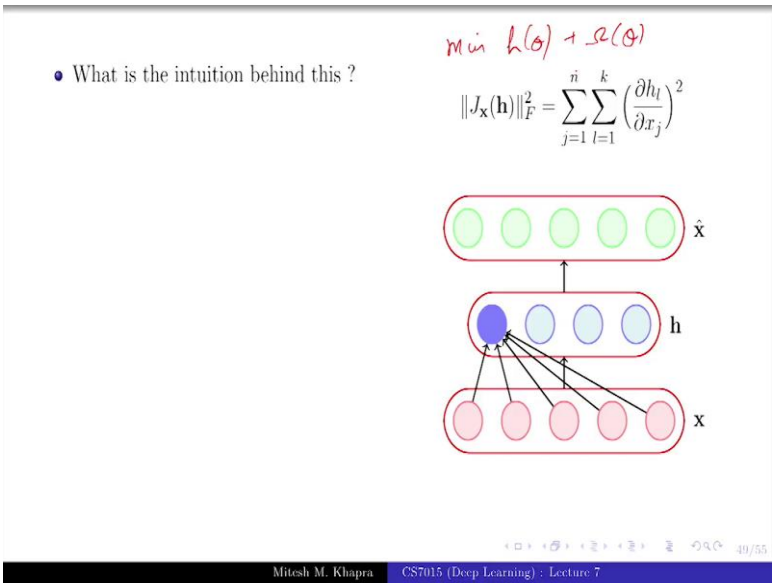
How much does h l change with a small change in.

Student: x k.

X k right, that is what a derivative captures is that fine and then what does the Frobenius norm capture, it is just the square of sum of the square of all the elements of the matrix

right? So, it is basically how much each of these elements vary with respect to the input and we are just taking the square of that. So, you see what is the term that we have added.

(Refer Slide Time: 02:44)



Now, tell me what is intuition behind this ok? So, when would this term; so, remember this term is added to the loss function and you are trying to minimize the loss function. So, that means, you want this term to go to.

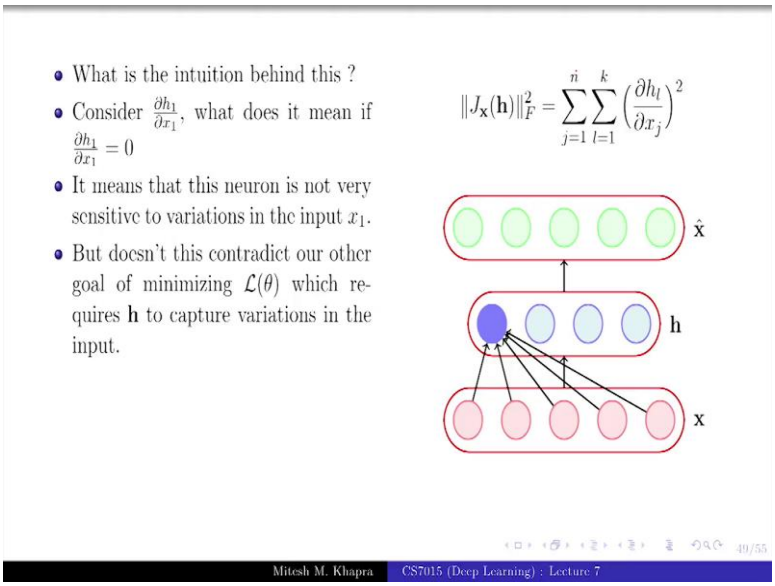
Student: (Refer Time: 03:01).

You want the Frobenius norm to be.

Student: 0 (Refer Time: 03:03).

0 right ideally of course, that will not happen because there is always a tradeoff between l theta and omega theta, if you make it 0 then l theta would be very high right?

(Refer Slide Time: 03:17)



So, now what would happen, if one of these guys say dou h 1 by dou x 1 actually goes to 0. What does that mean? h 1 is not sensitive to variations in x 1 right fine. But was our original mandate, what did we want these neurons to capture? We wanted the neurons to capture these important characteristics right?

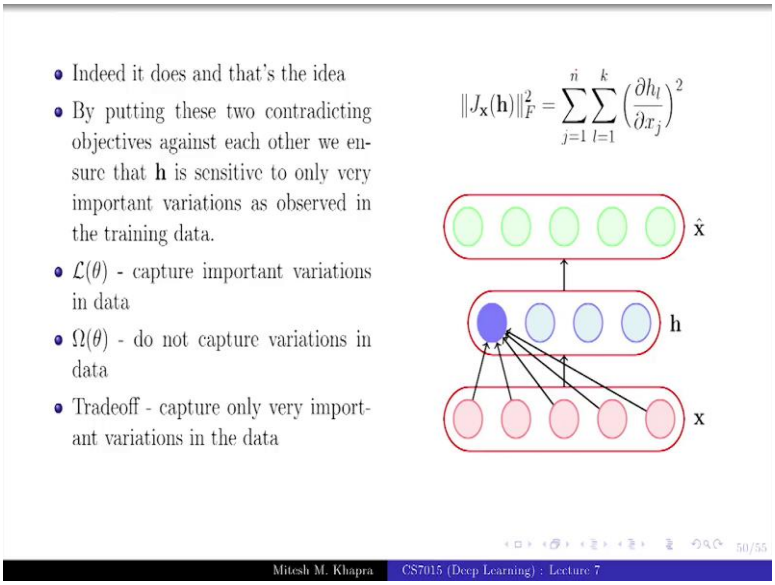
So, if x 1 changes we want h 1 to change, do you get that? How many of you get that? We wanted the neurons to capture the important characteristics of the data right? But now, we have added a contradictory condition which says that we do not want the neuron to capture a variations in the data, do you see this? So, what is happening here? L theta says that I should be able to capture these variations right? Otherwise I will not be able to reconstruct.

If all my h i's are not sensitive to variances x 1; that means, I give it any x 1 it will produces the same h i, is that clear is that with ok everyone right? That means, so see this is this. So, I have these training examples occurs all these training examples my bold x, which is vector x is going to change. That means, xi's which are the elements of this vectors are going to change.

Now, what this condition is saying is that if I change xi, I do not want the h l's to change, I do not want the values of the hidden representations to change. So;, that means, it is changing the respective of what is the input fed to it try to produce the same output, do you get this argument? Ok. That means, it is not capturing any important characteristics

of the data, is that fine is that valid argument, but that is not what we wanted, we wanted it to capture the important characteristic of the data. So, what are we trying to do now?

(Refer Slide Time: 05:01)



So, just I, it is hard for me to do evaluate what you have said, but just pay attention and see if that is correct you can judge it on your own, right? So, that is the actually the idea right we have put these two contradictory conditions with each other right, l theta says capture the important variances of the data. Omega theta says do not capture variations in the data, watch the tradeoff capture only very important variations in the data do not capture the variations which are not important. Can you relate this to something that you have seen before?

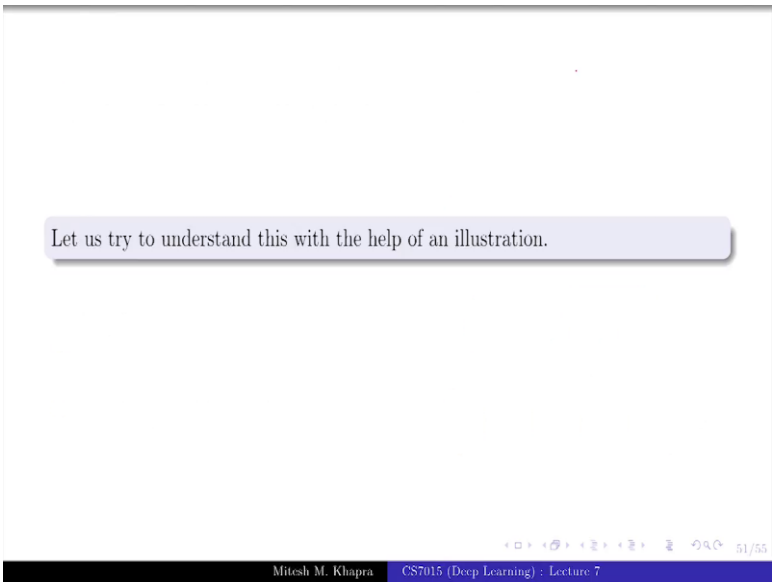
Student: Bias variance.

No, the other answer there are only two answers bias variance and PCA when I say the other answer.

Student: PCA.

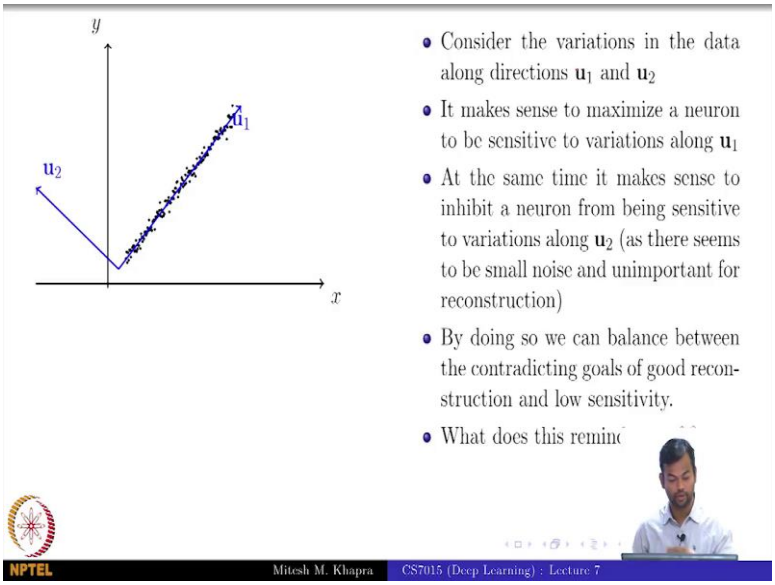
What am I trying to force it to do capture only the important variation, it is if it is not clear right now, we will come back to this ok.

(Refer Slide Time: 05:52)



So, let us try to understand with this with the help of an illustration right, how many of you get the argument which I made on this slide ok, most of all.

(Refer Slide Time: 06:03)



Now, this is the situation, I have u 1 and u 2 as my dimensions fine, which of this is important? u 1, the variations in the data across u 1 is something that I should care about. Because I can see that brings in some difference what about the variations in u 2.

Student: Not important.

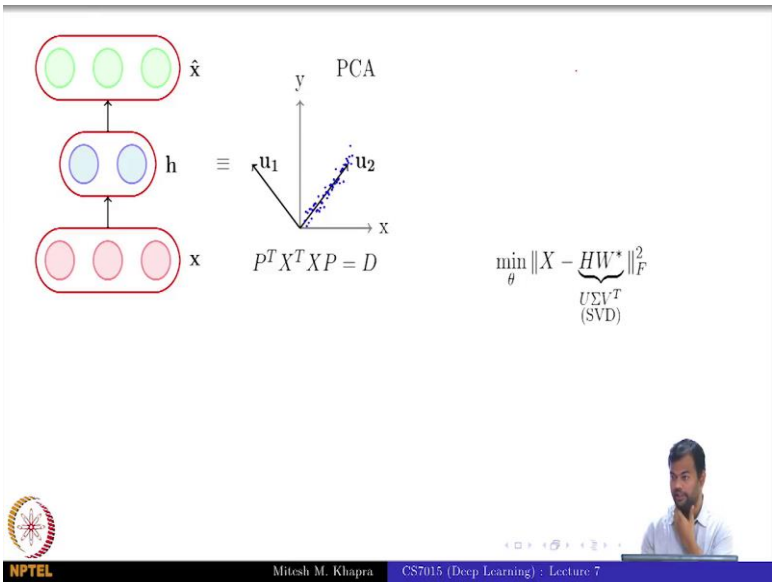
Not important, they seem like noises because these variations are there they are not all lying on the central line, they are slightly away from the line, here are some variations. But should I go out of my way to capture these variations, does it make sense to do that? No right? So, it makes sense to maximize a neuron to be sensitive to variations along u 1.

But it does not make sense to make the neuron sensitive to variations along this other dimension which is u 2 ok, by doing so we can balance the two conditions. So, one condition was trying to capture all the important variations ok do this, but do it only for the dimensions which really matter. The other conditions says that do not capture important variations ok do this, but do it only for those dimensions which do not matter. What is this remind you of? At least the diagram should have it away right.

Student: (Refer Time: 07:17)

It is same as principle component analysis right, so that is exactly what you try to do in PCA, you try to capture the variations across the important dimensions, but not across the non important dimensions. How many of you get the concept of contractive auto encoders? Ok good. So, I think that is a where we will end lecture 7.

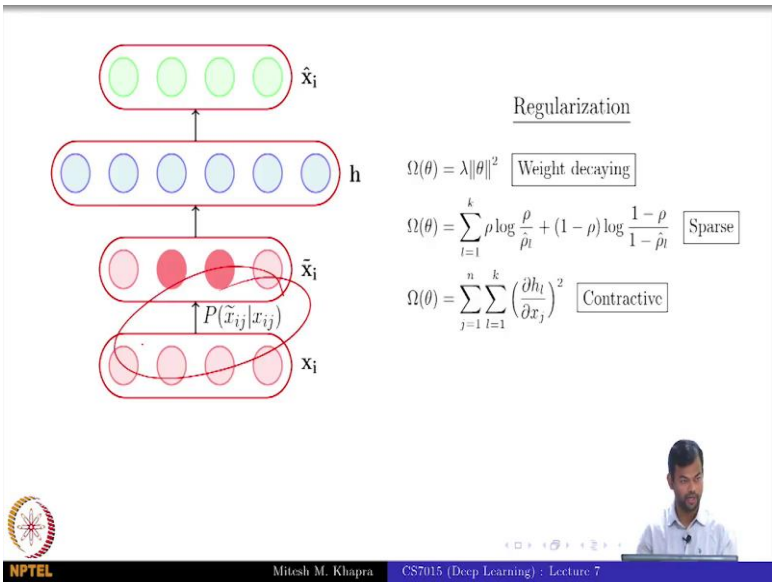
(Refer Slide Time: 07:38)



And just a quick summary; so, we showed that under certain conditions auto encoders are equivalent to PCA.

And we use this result very crucially there, that SVD theorem I will not state it.

(Refer Slide Time: 07:51)



And then we looked at different types of regularizations for auto encoders where we looked at weight decaying. That means, the standard L2 norm, we looked at the sparse auto encoder, the contractive auto encoder and we also looked at these denoising auto encoders right? So, that is the summary of this lecture.